

# COMSM1201 : Data Structures & Algorithms

Neill.Campbell@bristol.ac.uk

University of Bristol

Built : November 25, 2025



# Table of Contents

**N : Recursion**

O : Algorithms I - Search

P : Linked Data Structures

Q : ADTs - Collection

R : ADTs - Stacks

S : ADTs - Queues

T : ADTs - Trees

U : ADTs - Hashing

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

# Simple Recursion

- When a function calls itself, this is known as recursion.

# Simple Recursion

- When a function calls itself, this is known as recursion.
- This is an important theme in Computer Science that crops up time & time again.

# Simple Recursion

- When a function calls itself, this is known as recursion.
- This is an important theme in Computer Science that crops up time & time again.
- Can sometimes lead to very simple and elegant programs.

# Simple Recursion

- When a function calls itself, this is known as recursion.
- This is an important theme in Computer Science that crops up time & time again.
- Can sometimes lead to very simple and elegant programs.
- Let's look at some toy examples to begin with.

# Simple Recursion

- When a function calls itself, this is known as recursion.
- This is an important theme in Computer Science that crops up time & time again.
- Can sometimes lead to very simple and elegant programs.
- Let's look at some toy examples to begin with.

# Simple Recursion

- When a function calls itself, this is known as recursion.
- This is an important theme in Computer Science that crops up time & time again.
- Can sometimes lead to very simple and elegant programs.
- Let's look at some toy examples to begin with.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define SWAP(A,B) {char temp; temp=A;A=B;B=temp;}
5
6  void strrev(char* s, int n);
7
8  int main(void)
9  {
10     char str[] = "Hello World!";
11     strrev(str, strlen(str));
12     printf("%s\n", str);
13     return 0;
14 }
15
16 /* Iterative Inplace String Reverse */
17 void strrev(char* s, int n)
18 {
19     for(int i=0, j=n-1; i<j; i++, j--){
20         SWAP(s[i], s[j]);
21     }
22 }
```

Execution :

!dlroW olleH



# Recursion for *strrev()*

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define SWAP(A,B) {char temp; temp=A;A=B;B=temp;}
5
6  void strrev(char* s, int start, int end);
7
8  int main(void)
9  {
10     char str[] = "Hello World!";
11     strrev(str, 0, strlen(str)-1);
12     printf("%s\n", str);
13     return 0;
14 }
15
16 /* Recursive : Inplace String Reverse */
17 void strrev(char* s, int start, int end)
18 {
19     if(start >= end){
20         return;
21     }
22     SWAP(s[start], s[end]);
23     strrev(s, start+1, end-1);
24 }
```

Execution :

!dlroW olleH

# Recursion for *strrev()*

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define SWAP(A,B) {char temp; temp=A;A=B;B=temp;}
5
6  void strrev(char* s, int start, int end);
7
8  int main(void)
9  {
10     char str[] = "Hello World!";
11     strrev(str, 0, strlen(str)-1);
12     printf("%s\n", str);
13     return 0;
14 }
15
16 /* Recursive : Inplace String Reverse */
17 void strrev(char* s, int start, int end)
18 {
19     if(start >= end){
20         return;
21     }
22     SWAP(s[start], s[end]);
23     strrev(s, start+1, end-1);
24 }
```

Execution :

!dlroW olleH

- We need to change the function prototype.

# Recursion for *strrev()*

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define SWAP(A,B) {char temp; temp=A;A=B;B=temp;}
5
6  void strrev(char* s, int start, int end);
7
8  int main(void)
9  {
10     char str[] = "Hello World!";
11     strrev(str, 0, strlen(str)-1);
12     printf("%s\n", str);
13     return 0;
14 }
15
16 /* Recursive : Inplace String Reverse */
17 void strrev(char* s, int start, int end)
18 {
19     if(start >= end){
20         return;
21     }
22     SWAP(s[start], s[end]);
23     strrev(s, start+1, end-1);
24 }
```

Execution :

!dlroW olleH

- We need to change the function prototype.
- This allows us to track both the start and the end of the string.

# Recursion for *strrev()*

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define SWAP(A,B) {char temp; temp=A;A=B;B=temp;}
5
6  void strrev(char* s, int start, int end);
7
8  int main(void)
9  {
10     char str[] = "Hello World!";
11     strrev(str, 0, strlen(str)-1);
12     printf("%s\n", str);
13     return 0;
14 }
15
16 /* Recursive : Inplace String Reverse */
17 void strrev(char* s, int start, int end)
18 {
19     if(start >= end){
20         return;
21     }
22     SWAP(s[start], s[end]);
23     strrev(s, start+1, end-1);
24 }
```

Execution :

!dlroW olleH

- We need to change the function prototype.
- This allows us to track both the start and the end of the string.
- This is a form of 'tail-end' recursion.

# Recursion for *strrev()*

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define SWAP(A,B) {char temp; temp=A;A=B;B=temp;}
5
6  void strrev(char* s, int start, int end);
7
8  int main(void)
9  {
10     char str[] = "Hello World!";
11     strrev(str, 0, strlen(str)-1);
12     printf("%s\n", str);
13     return 0;
14 }
15
16 /* Recursive : Inplace String Reverse */
17 void strrev(char* s, int start, int end)
18 {
19     if(start >= end){
20         return;
21     }
22     SWAP(s[start], s[end]);
23     strrev(s, start+1, end-1);
24 }
```

Execution :

!dlroW olleH

- We need to change the function prototype.
- This allows us to track both the start and the end of the string.
- This is a form of 'tail-end' recursion.
- Some compilers can optimize for this since the current function can be terminated without waiting for the next function to return.

# The Fibonacci Sequence

A well known example of a recursive function is the Fibonacci sequence. The first term is 1, the second term is 1 and each successive term is defined to be the sum of the two previous terms, i.e. :

$\text{fib}(1)$  is 1

$\text{fib}(2)$  is 1

$\text{fib}(n)$  is  $\text{fib}(n-1) + \text{fib}(n-2)$

1, 1, 2, 3, 5, 8, 13, 21, ...

# Iterative & Recursive Fibonacci

```
1  #include <stdio.h>
2
3  #define MAXFIB 24
4
5  int fibonacci(int n);
6
7  int main(void)
8  {
9
10     for(int i=1; i<=MAXFIB; i++){
11         printf("%d = %d\n", i, fibonacci(i));
12     }
13
14     return 0;
15 }
16
17
18 int fibonacci(int n)
19 {
20     if(n <= 2){
21         return 1;
22     }
23     int a = 1;
24     int b = 1;
25     int next;
26     for(int i=3; i<=n; i++){
27         next = a + b;
28         a = b;
29         b = next;
30     }
31     return b;
32 }
```

# Iterative & Recursive Fibonacci

```
1  #include <stdio.h>
2
3  #define MAXFIB 24
4
5  int fibonacci(int n);
6
7  int main(void)
8  {
9
10     for(int i=1; i<=MAXFIB; i++){
11         printf("%d = %d\n", i, fibonacci(i));
12     }
13
14     return 0;
15 }
16
17 int fibonacci(int n)
18 {
19     if(n <= 2){
20         return 1;
21     }
22     int a = 1;
23     int b = 1;
24     int next;
25     for(int i=3; i<=n; i++){
26         next = a + b;
27         a = b;
28         b = next;
29     }
30     return b;
31 }
32 }
```

Execution :

```
1 = 1
2 = 1
3 = 2
4 = 3
5 = 5
6 = 8
7 = 13
8 = 21
9 = 34
10 = 55
11 = 89
12 = 144
13 = 233
14 = 377
15 = 610
16 = 987
17 = 1597
18 = 2584
19 = 4181
20 = 6765
21 = 10946
22 = 17711
23 = 28657
24 = 46368
```



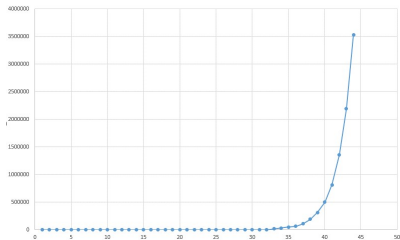
# Iterative & Recursive Fibonacci

```
1  #include <stdio.h>
2
3  #define MAXFIB 24
4
5  int fibonacci(int n);
6
7  int main(void)
8  {
9
10     for(int i=1; i<=MAXFIB; i++){
11         printf("%d = %d\n", i, fibonacci(i));
12     }
13
14     return 0;
15
16 }
17
18 int fibonacci(int n)
19 {
20     if(n == 1) return 1;
21     if(n == 2) return 1;
22     return( fibonacci(n-1)+fibonacci(n-2));
23 }
```

# Iterative & Recursive Fibonacci

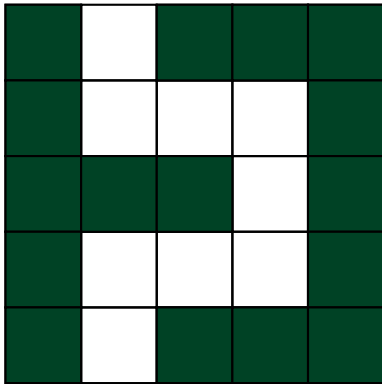
```
1  #include <stdio.h>
2
3  #define MAXFIB 24
4
5  int fibonacci(int n);
6
7  int main(void)
8  {
9
10     for(int i=1; i<=MAXFIB; i++){
11         printf("%d = %d\n", i, fibonacci(i));
12     }
13
14     return 0;
15 }
16
17
18 int fibonacci(int n)
19 {
20     if(n == 1) return 1;
21     if(n == 2) return 1;
22     return( fibonacci(n-1)+fibonacci(n-2));
23 }
```

It's interesting to see how run-time increases as the length of the sequence is raised.



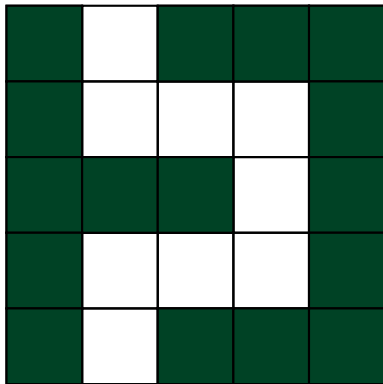
# Maze Escape

The correct route through a maze can be obtained via recursive, rather than iterative, methods.



# Maze Escape

The correct route through a maze can be obtained via recursive, rather than iterative, methods.



```
bool explore(int x, int y, char mz[YS][XS])
{
    if mz[y][x] is exit return true;

    Mark mz[y][x] so we don't return here

    if we can go up :
        if(explore(x, y+1, mz)) return true

    if we can go right :
        if(explore(x+1, y, mz)) return true

    Do left & down in a similar manner

    return false; // Failed to find route
}
```

# Permuting

- Here we consider the ways to permute a string (or more generally an array)

# Permuting

- Here we consider the ways to permute a string (or more generally an array)
- Permutations are all possible ways of rearranging the positions of the characters.

Execution :

ABC  
ACB  
BAC  
BCA  
CBA  
CAB

# Permuting

- Here we consider the ways to permute a string (or more generally an array)
- Permutations are all possible ways of rearranging the positions of the characters.

Execution :

ABC  
ACB  
BAC  
BCA  
CBA  
CAB

# Permuting

- Here we consider the ways to permute a string (or more generally an array)
- Permutations are all possible ways of rearranging the positions of the characters.

Execution :

ABC  
ACB  
BAC  
BCA  
CBA  
CAB

```
1 // From e.g. http://www.geeksforgeeks.org
2 #include <stdio.h>
3 #include <string.h>
4
5 #define SWAP(A,B) {char temp = *A; *A = *B; *B = temp;}
6
7 void permute(char* a, int s, int e);
8
9 int main()
10 {
11     char str[] = "ABC";
12     int n = strlen(str);
13     permute(str, 0, n-1);
14     return 0;
15 }
16
17 void permute(char* a, int s, int e)
18 {
19     if (s == e){
20         printf("%s\n", a);
21         return;
22     }
23     for (int i = s; i <= e; i++){
24         SWAP((a+s), (a+i)); // Bring one char to the front
25         permute(a, s+1, e);
26         SWAP((a+s), (a+i)); // Backtrack
27     }
28 }
```



# Self-test : Power

- Raising a number to a power  $n = 2^5$  is the same as multiple multiplications  
 $n = 2*2*2*2*2$ .

# Self-test : Power

- Raising a number to a power  $n = 2^5$  is the same as multiple multiplications  
 $n = 2 * 2 * 2 * 2 * 2$ .
- Or, thinking recursively,  $n = 2 * (2^4)$ .

# Self-test : Power

- Raising a number to a power  $n = 2^5$  is the same as multiple multiplications  
 $n = 2 * 2 * 2 * 2 * 2$ .
- Or, thinking recursively,  $n = 2 * (2^4)$ .

# Self-test : Power

- Raising a number to a power  $n = 2^5$  is the same as multiple multiplications  
 $n = 2*2*2*2*2$ .
- Or, thinking recursively,  $n = 2 * (2^4)$ .

```
1  /* Try to write power(a,b) to computer a^b
2  without using any maths functions other than
3  multiplication :
4  Try (1) iterative then (2) recursive
5  (3) Trick that for  $n\%2==0$ ,  $x^n = x^{(n/2)} * x^{(n/2)}$ 
6
7  */
8
9  #include <stdio.h>
10
11 int power(unsigned int a, unsigned int b);
12
13 int main(void)
14 {
15
16     int x = 2;
17     int y = 16;
18
19     printf("%d^%d = %d\n", x, y, power(x,y));
20
21 }
22
23 int power(unsigned int a, unsigned int b)
24 {
25 }
```

# Table of Contents

N : Recursion

**O : Algorithms I - Search**

P : Linked Data Structures

Q : ADTs - Collection

R : ADTs - Stacks

S : ADTs - Queues

T : ADTs - Trees

U : ADTs - Hashing

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

# Sequential Search

- The need to search an array for a particular value is a common problem.

# Sequential Search

- The need to search an array for a particular value is a common problem.
- This is used to delete names from a mailing list, or upgrading the salary of an employee etc.

# Sequential Search

- The need to search an array for a particular value is a common problem.
- This is used to delete names from a mailing list, or upgrading the salary of an employee etc.
- The simplest method for searching is called the sequential search.



# Sequential Search

- The need to search an array for a particular value is a common problem.
- This is used to delete names from a mailing list, or upgrading the salary of an employee etc.
- The simplest method for searching is called the sequential search.
- Simply move through the array from beginning to end, stopping when you have found the value you require.

# Sequential Search

- The need to search an array for a particular value is a common problem.
- This is used to delete names from a mailing list, or upgrading the salary of an employee etc.
- The simplest method for searching is called the sequential search.
- Simply move through the array from beginning to end, stopping when you have found the value you require.

# Sequential Search

- The need to search an array for a particular value is a common problem.
- This is used to delete names from a mailing list, or upgrading the salary of an employee etc.
- The simplest method for searching is called the sequential search.
- Simply move through the array from beginning to end, stopping when you have found the value you require.

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <assert.h>
4
5  #define NOTFOUND -1
6  #define NUMPEOPLE 6
7  typedef struct person{
8      char* name; int age;
9  } person;
10
11 int findAge(const char* name, const person* p, int n);
12
13 int main(void)
14 {
15     person ppl[NUMPEOPLE] = { {"Ackerby", 21}, {"Bloggs", 25},
16                               {"Chumley", 26}, {"Dalton", 25},
17                               {"Eggson", 22}, {"Fulton", 41} };
18
19     assert(findAge("Eggson", ppl, NUMPEOPLE)==22);
20     assert(findAge("Campbell", ppl, NUMPEOPLE)==NOTFOUND);
21     return 0;
22 }
23
24 int findAge(const char* name, const person* p, int n)
25 {
26     for(int j=0; j<n; j++){
27         if(strcmp(name, p[j].name) == 0){
28             return p[j].age;
29         }
30     }
31     return NOTFOUND;
32 }
```

# Sequential Search

- Sometimes our list of people may not be random.

# Sequential Search

- Sometimes our list of people may not be random.
- If, for instance, it is sorted, we can use `strcmp()` in a slightly cleverer manner.

# Sequential Search

- Sometimes our list of people may not be random.
- If, for instance, it is sorted, we can use `strcmp()` in a slightly cleverer manner.
- We can stop searching once the search key is alphabetically greater than the item at the current position in the list.

# Sequential Search

- Sometimes our list of people may not be random.
- If, for instance, it is sorted, we can use `strcmp()` in a slightly cleverer manner.
- We can stop searching once the search key is alphabetically greater than the item at the current position in the list.
- This halves, on average, the number of comparisons required.

# Sequential Search

- Sometimes our list of people may not be random.
- If, for instance, it is sorted, we can use `strcmp()` in a slightly cleverer manner.
- We can stop searching once the search key is alphabetically greater than the item at the current position in the list.
- This halves, on average, the number of comparisons required.



# Sequential Search

- Sometimes our list of people may not be random.
- If, for instance, it is sorted, we can use `strcmp()` in a slightly cleverer manner.
- We can stop searching once the search key is alphabetically greater than the item at the current position in the list.
- This halves, on average, the number of comparisons required.

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <assert.h>
4
5  #define NOTFOUND -1
6  #define NUMPEOPLE 6
7  typedef struct person{
8      char* name; int age;
9  } person;
10
11 int findAge(const char* name, const person* p, int n);
12
13 int main(void)
14 {
15     person ppl[NUMPEOPLE] = { {"Ackerby", 21}, {"Bloggs", 25},
16                               {"Chumley", 26}, {"Dalton", 25},
17                               {"Eggson", 22}, {"Fulton", 41} };
18
19     assert(findAge("Eggson", ppl, NUMPEOPLE)==22);
20     assert(findAge("Campbell", ppl, NUMPEOPLE)==NOTFOUND);
21     return 0;
22 }
23
24 int findAge(const char* name, const person* p, int n)
25 {
26     for(int j=0; j<n; j++){
27         int m = strcmp(name, p[j].name);
28         if(m == 0) // Braces!
29             return p[j].age;
30         if(m < 0)
31             return NOTFOUND;
32     }
33     return NOTFOUND;
34 }
```

# Binary Search for *101*

- Searching small lists doesn't require much computation time.

# Binary Search for *101*

- Searching small lists doesn't require much computation time.
- However, as lists get longer (e.g. phone directories), sequential searching becomes extremely inefficient.





# Binary Search for 101

- Searching small lists doesn't require much computation time.
- However, as lists get longer (e.g. phone directories), sequential searching becomes extremely inefficient.
- A binary search consists of examining the middle element of the array to see if it has the desired value. If not, then half the array may be discarded for the next search.

4    7    19    25    36    37    50    100    101    205    220    270    301    321

↑

↑

↑

↑

↑

↑

↑

↑

↑

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <assert.h>
4  #include <time.h>
5  #define NMBS 1000000
6
7  int bin_it(int k, const int* a, int l, int r);
8
9  int main(void)
10 {
11     int a[NMBS];
12     srand(time(NULL));
13
14     // Put even numbers into array
15     for(int i=0; i<NMBS; i++){
16         a[i] = 2*i;
17     }
18
19     // Do many searches for a random number
20     for(int i=0; i<10*NMBS; i++){
21         int n = rand()%NMBS;
22         if((n%2) == 0){
23             assert(bin_it(n, a, 0, NMBS-1) == n/2);
24         }
25         else{ // No odd numbers in this list
26             assert(bin_it(n, a, 0, NMBS-1) < 0);
27         }
28     }
29     return 0;
30 }
```

# Iterative v. Recursion Binary Search

```
int bin_it(int k, const int* a, int l, int r)
{
    while(l <= r){
        int m = (l+r)/2;
        if(k == a[m]){
            return m;
        }
        else{
            if (k > a[m]){
                l = m + 1;
            }
            else{
                r = m - 1;
            }
        }
    }
    return -1;
}
```

# Iterative v. Recursion Binary Search

```
int bin_it(int k, const int* a, int l, int r)
{
    while(l <= r){
        int m = (l+r)/2;
        if(k == a[m]){
            return m;
        }
        else{
            if (k > a[m]){
                l = m + 1;
            }
            else{
                r = m - 1;
            }
        }
    }
    return -1;
}
```

```
int bin_rec(int k, const int* a, int l, int r)
{
    if(l > r) return -1;
    int m = (l+r)/2;
    if(k == a[m]){
        return m;
    }
    else{
        if (k > a[m]){
            return bin_rec(k, a, m+1, r);
        }
        else{
            return bin_rec(k, a, l, m-1);
        }
    }
}
```



# Interpolation Search

- When we look for a word in a dictionary, we don't start in the middle. We make an educated guess as to where to start based on the 1st letter of the word being searched for.

# Interpolation Search

- When we look for a word in a dictionary, we don't start in the middle. We make an educated guess as to where to start based on the 1st letter of the word being searched for.
- This idea led to the interpolation search.

# Interpolation Search

- When we look for a word in a dictionary, we don't start in the middle. We make an educated guess as to where to start based on the 1st letter of the word being searched for.
- This idea led to the interpolation search.
- In binary searching, we simply used the middle of an ordered list as a best guess as to where to begin the search.

# Interpolation Search

- When we look for a word in a dictionary, we don't start in the middle. We make an educated guess as to where to start based on the 1st letter of the word being searched for.
- This idea led to the interpolation search.
- In binary searching, we simply used the middle of an ordered list as a best guess as to where to begin the search.
- Now we use an interpolation involving the key, the start of the list and the end.

$$i = (k - l[0]) / (l[n - 1] - l[0]) * n$$

# Interpolation Search

- When we look for a word in a dictionary, we don't start in the middle. We make an educated guess as to where to start based on the 1st letter of the word being searched for.
- This idea led to the interpolation search.
- In binary searching, we simply used the middle of an ordered list as a best guess as to where to begin the search.
- Now we use an interpolation involving the key, the start of the list and the end.

$$i = (k - l[0]) / (l[n - 1] - l[0]) * n$$

- when searching for '15' :

0   4   5   9   10   12   15   20  
                  ↑↑

# Interpolation Search

- When we look for a word in a dictionary, we don't start in the middle. We make an educated guess as to where to start based on the 1st letter of the word being searched for.
- This idea led to the interpolation search.
- In binary searching, we simply used the middle of an ordered list as a best guess as to where to begin the search.
- Now we use an interpolation involving the key, the start of the list and the end.

$$i = (k - l[0]) / (l[n - 1] - l[0]) * n$$

- when searching for '15' :

0   4   5   9   10   12   15   20  
                  ↑↑

# Interpolation Search

- When we look for a word in a dictionary, we don't start in the middle. We make an educated guess as to where to start based on the 1st letter of the word being searched for.
- This idea led to the interpolation search.
- In binary searching, we simply used the middle of an ordered list as a best guess as to where to begin the search.
- Now we use an interpolation involving the key, the start of the list and the end.

$$i = (k - l[0]) / (l[n - 1] - l[0]) * n$$

- when searching for '15' :

0   4   5   9   10   12   15   20  
          ↑↑

```
int interp(int k, const int* a, int l, int r)
{
    int m;
    double md;

    while(l <= r){
        md = ((double)(k-a[l])/
              (double)(a[r]-a[l]))*
              (double)(r-l)
              )
            +(double)(l);
        m = 0.5 + md;
        if((m > r) || (m < l)){
            return -1;
        }
        if(k == a[m])
            return m;
        else{
            if (k > a[m]){
                l = m + 1;
            }
            else{
                r = m - 1;
            }
        }
    }
}
```

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:



# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3
- But "wall-clock" time is generally not the thing that excites Computer Scientists.

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3
- But "wall-clock" time is generally not the thing that excites Computer Scientists.

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18
19 }
```

- Searching and sorting algorithms have a complexity associated with them, called big-O.

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3
- But "wall-clock" time is generally not the thing that excites Computer Scientists.

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- Searching and sorting algorithms have a complexity associated with them, called big-O.
- This complexity indicates how, for  $n$  numbers, performance deteriorates when  $n$  changes.

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3
- But "wall-clock" time is generally not the thing that excites Computer Scientists.

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3
- But "wall-clock" time is generally not the thing that excites Computer Scientists.

- Searching and sorting algorithms have a complexity associated with them, called big-O.
- This complexity indicates how, for  $n$  numbers, performance deteriorates when  $n$  changes.
- Sequential Search :  **$O(n)$**

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3
- But "wall-clock" time is generally not the thing that excites Computer Scientists.

- Searching and sorting algorithms have a complexity associated with them, called big-O.
- This complexity indicates how, for  $n$  numbers, performance deteriorates when  $n$  changes.
- Sequential Search :  **$O(n)$**
- Binary Search :  **$O(\log n)$**



# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3
- But "wall-clock" time is generally not the thing that excites Computer Scientists.

- Searching and sorting algorithms have a complexity associated with them, called big-O.
- This complexity indicates how, for  $n$  numbers, performance deteriorates when  $n$  changes.
- Sequential Search :  **$O(n)$**
- Binary Search :  **$O(\log n)$**
- Interpolation Search :  **$O(\log \log n)$**

# Algorithmic Complexity

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define CSEC (double)(CLOCKS_PER_SEC)
6  #define BIGLOOP 1000000000
7
8  int main(void)
9  {
10
11     clock_t c1 = clock();
12     for(int i=0; i<BIGLOOP; i++){
13         int j = i * 2;
14     }
15     clock_t c2 = clock();
16     printf("%f\n", (double)(c2-c1)/CSEC);
17     return 0;
18 }
19 }
```

- This code on an old Dell laptop took:
  - 3.12 seconds using a non-optimizing compiler -O0
  - 0.00 seconds using an aggressive optimization -O3
- But "wall-clock" time is generally not the thing that excites Computer Scientists.

- Searching and sorting algorithms have a complexity associated with them, called big-O.
- This complexity indicates how, for  $n$  numbers, performance deteriorates when  $n$  changes.
- Sequential Search :  **$O(n)$**
- Binary Search :  **$O(\log n)$**
- Interpolation Search :  **$O(\log \log n)$**
- We'll discuss the dream of a  **$O(1)$**  search later in "Hashing".

# Binary vs. Interpolation Timing

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <assert.h>
4  #include <time.h>
5
6  int bin_it(int k, const int *a, int l, int r);
7  int bin_rec(int k, const int *a, int l, int r);
8  int interp(int k, const int *a, int l, int r);
9  int* parse_args(int argc, char* argv[], int* n, int* srch);
10
11 int main(int argc, char* argv[])
12 {
13
14     int i, n, srch;
15     int* a;
16     int (*p[3])(int k, const int*a, int l, int r) =
17         {bin_it, bin_rec, interp};
18
19     a = parse_args(argc, argv, &n, &srch);
20
21     srand(time(NULL));
22     for(i=0; i<n; i++){
23         a[i] = 2*i;
24     }
25     for(i=0; i<5000000; i++){
26         assert((*p[srch])(a[rand()%n], a, 0, n-1) >= 0);
27     }
28
29     free(a);
30     return 0;
31 }
32 }
```

# Binary vs. Interpolation Timing

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <assert.h>
4  #include <time.h>
5
6  int bin_it(int k, const int *a, int l, int r);
7  int bin_rec(int k, const int *a, int l, int r);
8  int interp(int k, const int *a, int l, int r);
9  int* parse_args(int argc, char* argv[], int* n, int* srch);
10
11 int main(int argc, char* argv[])
12 {
13
14     int i, n, srch;
15     int* a;
16     int (*p[3])(int k, const int*a, int l, int r) =
17         {bin_it, bin_rec, interp};
18
19     a = parse_args(argc, argv, &n, &srch);
20
21     srand(time(NULL));
22     for(i=0; i<n; i++){
23         a[i] = 2*i;
24     }
25     for(i=0; i<5000000; i++){
26         assert((*p[srch])(a[rand()%n], a, 0, n-1) >= 0);
27     }
28
29     free(a);
30     return 0;
31 }
32 }
```

Execution :

Binary Search : Iterative

n = 100000 = 0.39

n = 800000 = 0.57

n = 6400000 = 1.00

n = 51200000 = 2.46

Binary Search : Recursive

n = 100000 = 0.40

n = 800000 = 0.56

n = 6400000 = 0.97

n = 51200000 = 2.42

Interpolation

n = 100000 = 0.05

n = 800000 = 0.05

n = 6400000 = 0.10

n = 51200000 = 0.13

# Table of Contents

N : Recursion

O : Algorithms I - Search

**P : Linked Data Structures**

Q : ADTs - Collection

R : ADTs - Stacks

S : ADTs - Queues

T : ADTs - Trees

U : ADTs - Hashing

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

# Linked Data Structures

- Linked data representations are useful when:

# Linked Data Structures

- Linked data representations are useful when:
  - It is difficult to predict the size and the shape of the data structures in advance.

# Linked Data Structures

- Linked data representations are useful when:
  - It is difficult to predict the size and the shape of the data structures in advance.
  - We need to efficiently insert and delete elements.



# Linked Data Structures

- Linked data representations are useful when:
  - It is difficult to predict the size and the shape of the data structures in advance.
  - We need to efficiently insert and delete elements.
- To create linked data representations we use pointers to connect separate blocks of storage together. If a given block contains a pointer to a second block, we can follow this pointer there.

# Linked Data Structures

- Linked data representations are useful when:
  - It is difficult to predict the size and the shape of the data structures in advance.
  - We need to efficiently insert and delete elements.
- To create linked data representations we use pointers to connect separate blocks of storage together. If a given block contains a pointer to a second block, we can follow this pointer there.
- By following pointers one after another, we can travel right along the structure.

# Linked Data Structures

- Linked data representations are useful when:
  - It is difficult to predict the size and the shape of the data structures in advance.
  - We need to efficiently insert and delete elements.
- To create linked data representations we use pointers to connect separate blocks of storage together. If a given block contains a pointer to a second block, we can follow this pointer there.
- By following pointers one after another, we can travel right along the structure.

# Linked Data Structures

- Linked data representations are useful when:
  - It is difficult to predict the size and the shape of the data structures in advance.
  - We need to efficiently insert and delete elements.
- To create linked data representations we use pointers to connect separate blocks of storage together. If a given block contains a pointer to a second block, we can follow this pointer there.
- By following pointers one after another, we can travel right along the structure.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "general.h"
4
5  typedef struct data{
6      int i;
7      struct data* next;
8  } Data;
9
10 Data* allocateData(int i);
11 void printList(Data* l);
12
13 int main(void)
14 {
15     int i;
16     Data* start, *current;
17     start = current = NULL;
18     printf("Enter the first number: ");
19     if(scanf("%i", &i) == 1){
20         start = current = allocateData(i);
21     }
22     else{
23         on_error("Couldn't read an int");
24     }
25
26     printf("Enter more numbers: ");
27     while(scanf("%i", &i) == 1){
28         current->next = allocateData(i);
29         current = current->next;
30     }
31     printList(start);
32     return 0; // Should Free List
33 }
```

# Linked Lists

```
Data* allocateData(int i)
{
    Data* p;
    p = (Data*) ncalloc(1, sizeof(Data));
    p->i = i;
    // Not really required
    p->next = NULL;
    return p;
}

void printList(Data* l)
{
    printf("\n");
    do{
        printf("Number : %i\n", l->i);
        l = l->next;
    }while(l != NULL);
    printf("END\n");
}
```

# Linked Lists

```
Data* allocateData(int i)
{
    Data* p;
    p = (Data*) calloc(1, sizeof(Data));
    p->i = i;
    // Not really required
    p->next = NULL;
    return p;
}

void printList(Data* l)
{
    printf("\n");
    do{
        printf("Number : %i\n", l->i);
        l = l->next;
    }while(l != NULL);
    printf("END\n");
}
```

## Searching and Recursive printing:

```
Data* inList(Data* n, int i)
{
    do{
        if(n->i==i){
            return n;
        }
        n = n->next;
    }while(n != NULL);
    return NULL;
}

void printList_r(Data* l)
{
    // Recursive Base-Case
    if(l == NULL) return;

    printf("Number: %i\n", l->i);
    printList_r(l->next);
}
```

# Abstract Data Types

- But would we really code something like this **every** time we need flexible data storage ?

# Abstract Data Types

- But would we really code something like this **every** time we need flexible data storage ?
- This would be horribly error-prone.



# Abstract Data Types

- But would we really code something like this **every** time we need flexible data storage ?
- This would be horribly error-prone.
- Build something once, and test it well.

# Abstract Data Types

- But would we really code something like this **every** time we need flexible data storage ?
- This would be horribly error-prone.
- Build something once, and test it well.
- One example of this is an **Abstract Data Type (ADT)**.

# Abstract Data Types

- But would we really code something like this **every** time we need flexible data storage ?
- This would be horribly error-prone.
- Build something once, and test it well.
- One example of this is an **Abstract Data Type (ADT)**.
- Each ADT exposes its functionality via an *interface*. The **user** only accesses the data via this interface.

# Abstract Data Types

- But would we really code something like this **every** time we need flexible data storage ?
- This would be horribly error-prone.
- Build something once, and test it well.
- One example of this is an **Abstract Data Type (ADT)**.
- Each ADT exposes its functionality via an *interface*. The **user** only accesses the data via this interface.
- The **user** of the ADT doesn't need to understand how the data is being stored (e.g. array vs. linked lists etc.)

# Abstract Data Types

- But would we really code something like this **every** time we need flexible data storage ?
- This would be horribly error-prone.
- Build something once, and test it well.
- One example of this is an **Abstract Data Type (ADT)**.
- Each ADT exposes its functionality via an *interface*. The **user** only accesses the data via this interface.
- The **user** of the ADT doesn't need to understand how the data is being stored (e.g. array vs. linked lists etc.)
- Actually, I'll sometimes blur the boundaries of Data Structures (e.g. a linked list) with ADTs (e.g. a dictionary) themselves.

# Table of Contents

N : Recursion

O : Algorithms I - Search

P : Linked Data Structures

**Q : ADTs - Collection**

R : ADTs - Stacks

S : ADTs - Queues

T : ADTs - Trees

U : ADTs - Hashing

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

- One of the simplest ADTs is the **Collection**.

# Collections

- One of the simplest ADTs is the **Collection**.
- This is just a simple place to search for/add/delete data elements.



- One of the simplest ADTs is the **Collection**.
- This is just a simple place to search for/add/delete data elements.
- Some collections allow duplicate elements and others do not (e.g. Sets).

- One of the simplest ADTs is the **Collection**.
- This is just a simple place to search for/add/delete data elements.
- Some collections allow duplicate elements and others do not (e.g. Sets).
- Some are ordered (for faster searching) and others unordered.

- One of the simplest ADTs is the **Collection**.
- This is just a simple place to search for/add/delete data elements.
- Some collections allow duplicate elements and others do not (e.g. Sets).
- Some are ordered (for faster searching) and others unordered.
- Our Collection will be unsorted and will allow duplicates.

- One of the simplest ADTs is the **Collection**.
- This is just a simple place to search for/add/delete data elements.
- Some collections allow duplicate elements and others do not (e.g. Sets).
- Some are ordered (for faster searching) and others unordered.
- Our Collection will be unsorted and will allow duplicates.

# Collections

- One of the simplest ADTs is the **Collection**.
- This is just a simple place to search for/add/delete data elements.
- Some collections allow duplicate elements and others do not (e.g. Sets).
- Some are ordered (for faster searching) and others unordered.
- Our Collection will be unsorted and will allow duplicates.

```
1  #include "../General/general.h"
2
3  typedef int colltype;
4
5  typedef struct coll coll;
6
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <assert.h>
10
11 // Create an empty coll
12 coll* coll_init(void);
13 // Add element onto top
14 void coll_add(coll* c, colltype i);
15 // Take element out
16 bool coll_remove(coll* c, colltype d);
17 // Does this exist ?
18 bool coll_isin(coll* c, colltype i);
19 // Return size of coll
20 int coll_size(coll* c);
21 // Clears all space used
22 bool coll_free(coll* c);
```

# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.

# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.
- A user of the ADT doesn't really need to know how it's implemented - ideally.

# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.
- A user of the ADT doesn't really need to know how it's implemented - ideally.
- The ADT developer could have several **different** implementations.



# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.
- A user of the ADT doesn't really need to know how it's implemented - ideally.
- The ADT developer could have several **different** implementations.
- Here we'll see *Collection* implemented using:

# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.
- A user of the ADT doesn't really need to know how it's implemented - ideally.
- The ADT developer could have several **different** implementations.
- Here we'll see *Collection* implemented using:
  - A fixed-size array

# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.
- A user of the ADT doesn't really need to know how it's implemented - ideally.
- The ADT developer could have several **different** implementations.
- Here we'll see *Collection* implemented using:
  - A fixed-size array
  - A dynamic array

# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.
- A user of the ADT doesn't really need to know how it's implemented - ideally.
- The ADT developer could have several **different** implementations.
- Here we'll see *Collection* implemented using:
  - A fixed-size array
  - A dynamic array
  - A linked-list

# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.
- A user of the ADT doesn't really need to know how it's implemented - ideally.
- The ADT developer could have several **different** implementations.
- Here we'll see *Collection* implemented using:
  - A fixed-size array
  - A dynamic array
  - A linked-list

# Collection ADT

- Note that the interface gives you no hints as to the actual underlying implementation of the ADT.
- A user of the ADT doesn't really need to know how it's implemented - ideally.
- The ADT developer could have several **different** implementations.
- Here we'll see *Collection* implemented using:
  - A fixed-size array
  - A dynamic array
  - A linked-list

Fixed/specific.h:

```
1  #pragma once
2
3  #define COLTYPE "Fixed"
4
5  #define FIXEDSIZE 5000
6  struct coll {
7      // Underlying array
8      colltype a[FIXEDSIZE];
9      int size;
10 };
```

# Collection ADT using a Fixed-size Array

Fixed/fixed.c:

```
1  #include "../coll.h"
2  #include "specific.h"
3
4  coll* coll_init(void)
5  {
6      coll* c = (coll*) ncalloc(1, sizeof(coll));
7      c->size = 0;
8      return c;
9  }
10
11 int coll_size(coll* c)
12 {
13     if(c==NULL){
14         return 0;
15     }
16     return c->size;
17 }
18
19 bool coll_isin(coll* c, colltype d)
20 {
21     for(int i=0; i<coll_size(c); i++){
22         if(c->a[i] == d){
23             return true;
24         }
25     }
26     return false;
27 }
```

# Collection ADT using a Fixed-size Array

## Fixed/fixed.c:

```
1  #include "../coll.h"
2  #include "specific.h"
3
4  coll* coll_init(void)
5  {
6      coll* c = (coll*) ncalloc(1, sizeof(coll));
7      c->size = 0;
8      return c;
9  }
10
11 int coll_size(coll* c)
12 {
13     if(c==NULL){
14         return 0;
15     }
16     return c->size;
17 }
18
19 bool coll_isin(coll* c, colltype d)
20 {
21     for(int i=0; i<coll_size(c); i++){
22         if(c->a[i] == d){
23             return true;
24         }
25     }
26     return false;
27 }
```

```
void coll_add(coll* c, colltype d)
{
    if(c){
        if(c->size >= FIXEDSIZE){
            on_error("Collection overflow");
        }
        c->a[c->size] = d;
        c->size = c->size + 1;
    }
}

bool coll_remove(coll* c, colltype d)
{
    for(int i=0; i<coll_size(c); i++){
        if(c->a[i] == d){
            // Shuffle end of array left one
            for(int j=i; j<coll_size(c); j++){
                c->a[j] = c->a[j+1];
            }
            c->size = c->size - 1;
            return true;
        }
    }
    return false;
}

bool coll_free(coll* c)
{
    free(c);
    return true;
}
```



# Collection ADT via an Array (Realloc)

Realloc/specific.h:

```
1  #pragma once
2
3  #define COLTYPE "Realloc"
4
5  #define INITSIZE 16
6  #define SCALEFACTOR 2
7  struct coll {
8      // Underlying array
9      colltype* a;
10     int size;
11     int capacity;
12 };
```

# Collection ADT via an Array (Realloc)

## Realloc/specific.h:

```
1  #pragma once
2
3  #define COLLTTYPE "Realloc"
4
5  #define INITSIZE 16
6  #define SCALEFACTOR 2
7  struct coll {
8      // Underlying array
9      colltype* a;
10     int size;
11     int capacity;
12 };
```

## Realloc/realloc.c:

```
1  #include "../coll.h"
2  #include "specific.h"
3
4  coll* coll_init(void)
5  {
6      coll* c = (coll*) ncalloc(1, sizeof(coll));
7      c->a = (colltype*) ncalloc(INITSIZE, sizeof(colltype));
8      c->size = 0;
9      c->capacity = INITSIZE;
10     return c;
11 }
12
13 void coll_add(coll* c, colltype d)
14 {
15     if(c){
16         if(c->size >= c->capacity){
17             c->a = (colltype*) nrealloc(c->a,
18                                     sizeof(colltype)*c->capacity*SCALEFACTOR);
19             c->capacity = c->capacity*SCALEFACTOR;
20         }
21         c->a[c->size] = d;
22         c->size = c->size + 1;
23     }
24 }
```

# Collection ADT via a Linked List

Linked/specific.h:

```
1  #pragma once
2
3  #define COLLTYP "Linked"
4
5  struct dataframe {
6      colltype i;
7      struct dataframe* next;
8  };
9  typedef struct dataframe dataframe;
10
11 struct coll {
12     // Underlying array
13     dataframe* start;
14     int size;
15 };
```

# Collection ADT via a Linked List

## Linked/specific.h:

```
1  #pragma once
2
3  #define COLLTTYPE "Linked"
4
5  struct dataframe {
6      colltype i;
7      struct dataframe* next;
8  };
9  typedef struct dataframe dataframe;
10
11 struct coll {
12     // Underlying array
13     dataframe* start;
14     int size;
15 };
```

## Linked/linked.c:

```
#include "../coll.h"
#include "specific.h"

coll* coll_init(void)
{
    coll* c = (coll*) nalloc(1, sizeof(coll));
    return c;
}

int coll_size(coll* c)
{
    if(c==NULL){
        return 0;
    }
    return c->size;
}

bool coll_isin(coll* c, colltype d)
{
    if(c == NULL || c->start==NULL){
        return false;
    }
    dataframe* f = c->start;
    do{
        if(f->i == d){
            return true;
        }
        f = f->next;
    }while(f != NULL);
    return false;
}
```

# Collection ADT via a Linked List II

```
void coll_add(coll* c, colltype d)
{
    if(c){
        dataframe* f = nalloc(1, sizeof(dataframe));
        f->i = d;
        f->next = c->start;
        c->start = f;
        c->size = c->size + 1;
    }
}

bool coll_free(coll* c)
{
    if(c){
        dataframe* tmp;
        dataframe* p = c->start;
        while(p!=NULL){
            tmp = p->next;
            free(p);
            p = tmp;
        }
        free(c);
    }
    return true;
}
```

# Collection ADT via a Linked List II

```
void coll_add(coll* c, colltype d)
{
    if(c){
        dataframe* f = nalloc(1, sizeof(dataframe));
        f->i = d;
        f->next = c->start;
        c->start = f;
        c->size = c->size + 1;
    }
}

bool coll_free(coll* c)
{
    if(c){
        dataframe* tmp;
        dataframe* p = c->start;
        while(p!=NULL){
            tmp = p->next;
            free(p);
            p = tmp;
        }
        free(c);
    }
    return true;
}
```

```
bool coll_remove(coll* c, colltype d)
{
    dataframe* f1, *f2;
    if((c==NULL) || (c->start==NULL)){
        return false;
    }

    // If Front
    if(c->start->i == d){
        f1 = c->start->next;
        free(c->start);
        c->start = f1;
        c->size = c->size - 1;
        return true;
    }

    f1 = c->start;
    f2 = c->start->next;
    do{
        if(f2->i == d){
            f1->next = f2->next;
            free(f2);
            c->size = c->size - 1;
            return true;
        }
        f1 = f2;
        f2 = f1->next;
    }while(f2 != NULL);
    return false;
}
```

# Collection Summary

- Any code using the ADT can be compiled against any of the implementations, e.g. the test (`testcoll.c`) code.

# Collection Summary

- Any code using the ADT can be compiled against any of the implementations, e.g. the test (testcoll.c) code.
- The *Collection* interface (coll.h) is never changed.



# Collection Summary

- Any code using the ADT can be compiled against any of the implementations, e.g. the test (testcoll.c) code.
- The *Collection* interface (coll.h) is never changed.
- There are pros and cons of each implementation:

# Collection Summary

- Any code using the ADT can be compiled against any of the implementations, e.g. the test (testcoll.c) code.
- The *Collection* interface (coll.h) is never changed.
- There are pros and cons of each implementation:
  - Fixed Array : Simple to implement - can't avoid the problems of it being a fixed-size. Deletion expensive.

# Collection Summary

- Any code using the ADT can be compiled against any of the implementations, e.g. the test (testcoll.c) code.
- The *Collection* interface (coll.h) is never changed.
- There are pros and cons of each implementation:
  - Fixed Array : Simple to implement - can't avoid the problems of it being a fixed-size. Deletion expensive.
  - Realloc Array : Implementation fairly simple. Deletion expensive. Every realloc() is very **expensive**. Need to tune SCALEFACTOR.

# Collection Summary

- Any code using the ADT can be compiled against any of the implementations, e.g. the test (testcoll.c) code.
- The *Collection* interface (coll.h) is never changed.
- There are pros and cons of each implementation:
  - Fixed Array : Simple to implement - can't avoid the problems of it being a fixed-size. Deletion expensive.
  - Realloc Array : Implementation fairly simple. Deletion expensive. Every realloc() is very **expensive**. Need to tune SCALEFACTOR.
  - Linked : Slightly fiddly implementation
    - fast to delete an element.

# Collection Summary

- Any code using the ADT can be compiled against any of the implementations, e.g. the test (testcoll.c) code.
- The *Collection* interface (coll.h) is never changed.
- There are pros and cons of each implementation:
  - Fixed Array : Simple to implement - can't avoid the problems of it being a fixed-size. Deletion expensive.
  - Realloc Array : Implementation fairly simple. Deletion expensive. Every realloc() is very **expensive**. Need to tune SCALEFACTOR.
  - Linked : Slightly fiddly implementation
    - fast to delete an element.

# Collection Summary

- Any code using the ADT can be compiled against any of the implementations, e.g. the test (testcoll.c) code.
- The *Collection* interface (coll.h) is never changed.
- There are pros and cons of each implementation:
  - Fixed Array : Simple to implement - can't avoid the problems of it being a fixed-size. Deletion expensive.
  - Realloc Array : Implementation fairly simple. Deletion expensive. Every realloc() is very **expensive**. Need to tune SCALEFACTOR.
  - Linked : Slightly fiddly implementation - fast to delete an element.

| Task                  | Fixed Array                       | Realloc Array                         | Linked List                           |
|-----------------------|-----------------------------------|---------------------------------------|---------------------------------------|
| Insert new element    | $O(1)$ at end<br><i>if space</i>  | $O(1)$ at end<br><i>but realloc()</i> | $O(1)$ at front                       |
| Search for an element | $O(n)$<br><i>brute force</i>      | $O(n)$<br><i>brute force</i>          | $O(n)$<br><i>brute force</i>          |
| Search + delete       | $O(n) + O(n)$<br><i>move left</i> | $O(n) + O(n)$<br><i>move left</i>     | $O(n) + O(1)$<br><i>delete 'free'</i> |

- If we had ordered our ADT (ie. the elements were sorted), then the searches could be via a binary / interpolation search, leading to  $O(\log n)$  or  $O(\log \log n)$  search times.

# ADTs Making Coding Simpler

Linked List code from the previous Chapter :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "general.h"
4
5  typedef struct data{
6      int i;
7      struct data* next;
8  } Data;
9
10 Data* allocateData(int i);
11 void printList(Data* l);
12
13 int main(void)
14 {
15     int i;
16     Data* start, *current;
17     start = current = NULL;
18     printf("Enter the first number: ");
19     if(scanf("%i", &i) == 1){
20         start = current = allocateData(i);
21     }
22     else{
23         on_error("Couldn't read an int");
24     }
25
26     printf("Enter more numbers: ");
27     while(scanf("%i", &i) == 1){
28         current->next = allocateData(i);
29         current = current->next;
30     }
31     printList(start);
32     return 0; // Should Free List
33 }
```

# ADTs Making Coding Simpler

Linked List code from the previous Chapter :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "general.h"
4
5  typedef struct data{
6      int i;
7      struct data* next;
8  } Data;
9
10 Data* allocateData(int i);
11 void printList(Data* l);
12
13 int main(void)
14 {
15     int i;
16     Data* start, *current;
17     start = current = NULL;
18     printf("Enter the first number: ");
19     if(scanf("%i", &i) == 1){
20         start = current = allocateData(i);
21     }
22     else{
23         on_error("Couldn't read an int");
24     }
25
26     printf("Enter more numbers: ");
27     while(scanf("%i", &i) == 1){
28         current->next = allocateData(i);
29         current = current->next;
30     }
31     printList(start);
32     return 0; // Should Free List
33 }
```

Becomes :

```
1  #include "coll.h"
2  #include "Fixed/specific.h"
3
4  int main(void)
5  {
6      printf("Please type some numbers :");
7      coll* c = coll_init();
8      int i;
9      while(scanf("%i", &i) == 1){
10         coll_add(c, i);
11     }
12     // Do print etc.
13     coll_free(c);
14     return 0;
15 }
```



# Table of Contents

N : Recursion

O : Algorithms I - Search

P : Linked Data Structures

Q : ADTs - Collection

**R : ADTs - Stacks**

S : ADTs - Queues

T : ADTs - Trees

U : ADTs - Hashing

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

- Collections (Lists)

At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

- Collections (Lists)
- Stacks

At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

- Collections (Lists)
- Stacks
- Queues

At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

- Collections (Lists)
- Stacks
- Queues
- Sets

At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

- Collections (Lists)
- Stacks
- Queues
- Sets
- Graphs

At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

- Collections (Lists)
- Stacks
- Queues
- Sets
- Graphs
- Trees

At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

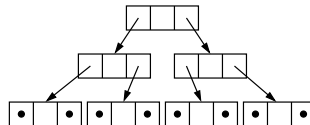
- Collections (Lists)
- Stacks
- Queues
- Sets
- Graphs
- Trees



At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

- Collections (Lists)
- Stacks
- Queues
- Sets
- Graphs
- Trees

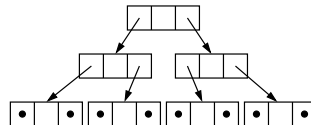
Binary Trees:



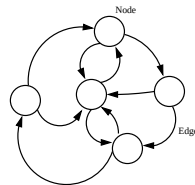
At the highest level of abstraction, ADTs that we can represent using both dynamic structures (pointers) and also fixed structures (arrays) include:

- Collections (Lists)
- Stacks
- Queues
- Sets
- Graphs
- Trees

Binary Trees:

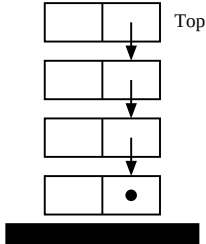


Unidirectional Graph:



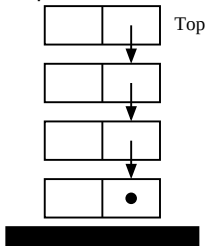
# Stacks

The push-down stack:

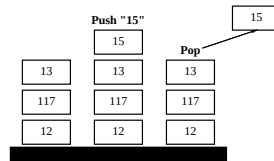


# Stacks

The push-down stack:



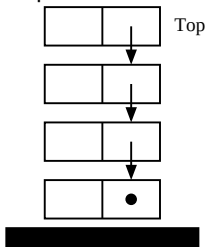
LIFO (Last in, First out):



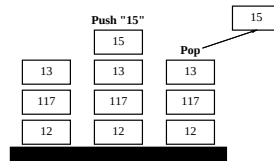
- Operations include push and pop.

# Stacks

The push-down stack:



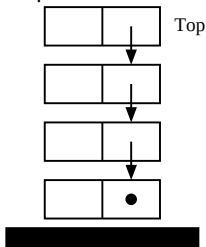
LIFO (Last in, First out):



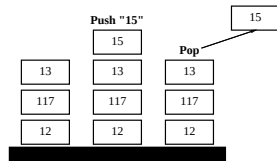
- Operations include push and pop.
- In the C run-time system, function calls are implemented using stacks.

# Stacks

The push-down stack:

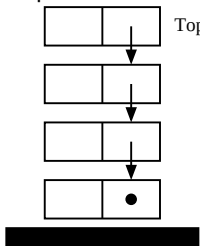


LIFO (Last in, First out):

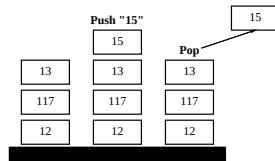


- Operations include push and pop.
- In the C run-time system, function calls are implemented using stacks.
- Most recursive algorithms can be re-written using stacks instead.

The push-down stack:



LIFO (Last in, First out):



- Operations include push and pop.
- In the C run-time system, function calls are implemented using stacks.
- Most recursive algorithms can be re-written using stacks instead.
- But, once again, we are faced with the question : How best to implement such a data type ?

# ADT:Stacks Arrays (Realloc) I

stack.h:

```
1  #pragma once
2
3  #include "../General/general.h"
4
5  typedef int stacktype;
6
7  typedef struct stack stack;
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13
14 /* Create an empty stack */
15 stack* stack_init(void);
16 /* Add element to top */
17 void stack_push(stack* s, stacktype i);
18 /* Take element from top */
19 bool stack_pop(stack* s, stacktype* d);
20 /* Clears all space used */
21 bool stack_free(stack* s);
22
23 /* Optional? */
24
25 /* Copy top element into d (but don't pop it) */
26 bool stack_peek(stack* s, stacktype* d);
27 /* Make a string version - keep .dot in mind */
28 void stack_tostring(stack* s, char* str);
```



# ADT:Stacks Arrays (Realloc) I

## stack.h:

```
1  #pragma once
2
3  #include "../General/general.h"
4
5  typedef int stacktype;
6
7  typedef struct stack stack;
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13
14 /* Create an empty stack */
15 stack* stack_init(void);
16 /* Add element to top */
17 void stack_push(stack* s, stacktype i);
18 /* Take element from top */
19 bool stack_pop(stack* s, stacktype* d);
20 /* Clears all space used */
21 bool stack_free(stack* s);
22
23 /* Optional? */
24
25 /* Copy top element into d (but don't pop it) */
26 bool stack_peek(stack* s, stacktype* d);
27 /* Make a string version - keep .dot in mind */
28 void stack_tostring(stack* s, char* str);
```

## Realloc/specific.h:

```
1  #pragma once
2
3  #define FORMATSTR "%i"
4  #define ELEMSIZE 20
5
6  #define STACKTYPE "Realloc"
7
8  #define FIXEDSIZE 16
9  #define SCALEFACTOR 2
10
11 struct stack {
12     /* Underlying array */
13     stacktype* a;
14     int size;
15     int capacity;
16 };
```

# ADT:Stacks Arrays (Realloc) II

## Realloc/realloc.c

```
1  #include "../stack.h"
2  #include "specific.h"
3
4  #define DOTFILE 5000
5
6  stack* stack_init(void)
7  {
8      stack *s = (stack*) ncalloc(1, sizeof(stack));
9      /* Some implementations would allow you to pass
10       a hint about the initial size of the stack */
11      s->a = (stacktype*) ncalloc(FIXEDSIZE, sizeof(stacktype));
12      s->size = 0;
13      s->capacity = FIXEDSIZE;
14      return s;
15  }
16
17 void stack_push(stack* s, stacktype d)
18 {
19     if(s==NULL){
20         return;
21     }
22     if(s->size >= s->capacity){
23         s->a = (stacktype*) nrealloc(s->a,
24                                     sizeof(stacktype)*s->capacity*SCALEFACTOR);
25         s->capacity = s->capacity*SCALEFACTOR;
26     }
27     s->a[s->size] = d;
28     s->size = s->size + 1;
29 }
```

# ADT:Stacks Arrays (Realloc) II

## Realloc/realloc.c

```
1  #include "../stack.h"
2  #include "specific.h"
3
4  #define DOTFILE 5000
5
6  stack* stack_init(void)
7  {
8      stack *s = (stack*) ncalloc(1, sizeof(stack));
9      /* Some implementations would allow you to pass
10       a hint about the initial size of the stack */
11      s->a = (stacktype*) ncalloc(FIXEDSIZE, sizeof(stacktype));
12      s->size = 0;
13      s->capacity = FIXEDSIZE;
14      return s;
15  }
16
17 void stack_push(stack* s, stacktype d)
18 {
19     if(s==NULL){
20         return;
21     }
22     if(s->size >= s->capacity){
23         s->a = (stacktype*) nrealloc(s->a,
24                                     sizeof(stacktype)*s->capacity*SCALEFACTOR);
25         s->capacity = s->capacity*SCALEFACTOR;
26     }
27     s->a[s->size] = d;
28     s->size = s->size + 1;
29 }
```

```
1  bool stack_pop(stack* s, stacktype* d)
2  {
3      if((s == NULL) || (s->size < 1)){
4          return false;
5      }
6      s->size = s->size - 1;
7      *d = s->a[s->size];
8      return true;
9  }
10
11 bool stack_peek(stack* s, stacktype* d)
12 {
13     if((s==NULL) || (s->size <= 0)){
14         /* Stack is Empty */
15         return false;
16     }
17     *d = s->a[s->size - 1];
18     return true;
19 }
```

# ADT:Stacks Arrays (Realloc) III

## Realloc/realloc.c

```
1 void stack_tostring(stack* s, char* str)
2 {
3     char tmp[ELEMSIZE];
4     str[0] = '\0';
5     if((s==NULL) || (s->size <1)){
6         return;
7     }
8     for(int i=s->size-1; i>=0; i--){
9         sprintf(tmp, FORMATSTR, s->a[i]);
10        strcat(str, tmp);
11        strcat(str, "|");
12    }
13    str[strlen(str)-1] = '\0';
14 }
15
16 bool stack_free(stack* s)
17 {
18     if(s==NULL){
19         return true;
20     }
21     free(s->a);
22     free(s);
23     return true;
24 }
```

# ADT:Stacks Arrays (Realloc) III

## Realloc/realloc.c

```
1 void stack_tostring(stack* s, char* str)
2 {
3     char tmp[ELEMSIZE];
4     str[0] = '\0';
5     if((s==NULL) || (s->size <1)){
6         return;
7     }
8     for(int i=s->size-1; i>=0; i--){
9         sprintf(tmp, FORMATSTR, s->a[i]);
10        strcat(str, tmp);
11        strcat(str, "|");
12    }
13    str[strlen(str)-1] = '\0';
14 }
15
16 bool stack_free(stack* s)
17 {
18     if(s==NULL){
19         return true;
20     }
21     free(s->a);
22     free(s);
23     return true;
24 }
```

- We need a thorough testing program teststack.c

# ADT:Stacks Arrays (Realloc) III

## Realloc/realloc.c

```
1 void stack_tostring(stack* s, char* str)
2 {
3     char tmp[ELEMSIZE];
4     str[0] = '\0';
5     if((s==NULL) || (s->size <1)){
6         return;
7     }
8     for(int i=s->size-1; i>=0; i--){
9         sprintf(tmp, FORMATSTR, s->a[i]);
10        strcat(str, tmp);
11        strcat(str, "|");
12    }
13    str[strlen(str)-1] = '\0';
14 }
15
16 bool stack_free(stack* s)
17 {
18     if(s==NULL){
19         return true;
20     }
21     free(s->a);
22     free(s);
23     return true;
24 }
```

- We need a thorough testing program teststack.c
- See also revstr.c : a version of the string reverse code (for which we already seen an iterative (in-place) and a recursive solution).

# ADT:Stacks Linked I

## Linked/specific.h

```
1  #pragma once
2
3  #define FORMATSTR "%i"
4  #define ELEMSIZE 20
5  #define STACKTYPE "Linked"
6
7  struct dataframe {
8      stacktype i;
9      struct dataframe* next;
10 };
11 typedef struct dataframe dataframe;
12
13 struct stack {
14     /* Underlying array */
15     dataframe* start;
16     int size;
17 };
```

# ADT:Stacks Linked I

## Linked/specific.h

```
1  #pragma once
2
3  #define FORMATSTR "%i"
4  #define ELEMSIZE 20
5  #define STACKTYPE "Linked"
6
7  struct dataframe {
8      stacktype i;
9      struct dataframe* next;
10 };
11 typedef struct dataframe dataframe;
12
13 struct stack {
14     /* Underlying array */
15     dataframe* start;
16     int size;
17 };
```

## Linked/linked.c

```
1  #include "../stack.h"
2  #include "specific.h"
3
4  #define DOTFILE 5000
5
6  stack* stack_init(void)
7  {
8      stack* s = (stack*) nalloc(1, sizeof(stack));
9      return s;
10 }
11
12 void stack_push(stack* s, stacktype d)
13 {
14     if(s){
15         dataframe* f = nalloc(1, sizeof(dataframe));
16         f->i = d;
17         f->next = s->start;
18         s->start = f;
19         s->size = s->size + 1;
20     }
21 }
```



# ADT:Stacks Linked II

```
1  bool stack_pop(stack* s, stacktype* d)
2  {
3      if((s==NULL) || (s->start==NULL)){
4          return false;
5      }
6
7      dataframe* f = s->start->next;
8      *d = s->start->i;
9      free(s->start);
10     s->start = f;
11     s->size = s->size - 1;
12     return true;
13 }
14
15 bool stack_peek(stack* s, stacktype* d)
16 {
17     if((s==NULL) || (s->start==NULL)){
18         return false;
19     }
20     *d = s->start->i;
21     return true;
22 }
```

# ADT:Stacks Linked II

```
1  bool stack_pop(stack* s, stacktype* d)
2  {
3      if((s==NULL) || (s->start==NULL)){
4          return false;
5      }
6
7      dataframe* f = s->start->next;
8      *d = s->start->i;
9      free(s->start);
10     s->start = f;
11     s->size = s->size - 1;
12     return true;
13 }
14
15 bool stack_peek(stack* s, stacktype* d)
16 {
17     if((s==NULL) || (s->start==NULL)){
18         return false;
19     }
20     *d = s->start->i;
21     return true;
22 }
```

```
1  void stack_tostring(stack* s, char* str)
2  {
3      char tmp[ELEMSIZE];
4      str[0] = '\0';
5      if((s==NULL) || (s->size < 1)){
6          return;
7      }
8      dataframe* p = s->start;
9      while(p){
10         sprintf(tmp, FORMATSIR, p->i);
11         strcat(str, tmp);
12         strcat(str, "|");
13         p = p->next;
14     }
15     str[strlen(str)-1] = '\0';
16 }
17
18 bool stack_free(stack* s)
19 {
20     if(s){
21         dataframe* p = s->start;
22         while(p!=NULL){
23             dataframe* tmp = p->next;
24             free(p);
25             p = tmp;
26         }
27         free(s);
28     }
29     return true;
30 }
```

# Table of Contents

N : Recursion

O : Algorithms I - Search

P : Linked Data Structures

Q : ADTs - Collection

R : ADTs - Stacks

**S : ADTs - Queues**

T : ADTs - Trees

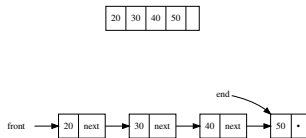
U : ADTs - Hashing

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

# ADTs : Queues

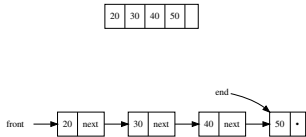
FIFO (First in, First out):



- Intuitively more “useful” than a stack.

# ADTs : Queues

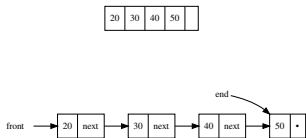
FIFO (First in, First out):



- Intuitively more “useful” than a stack.
- Think of implementing any kind of service (printer, web etc.)

# ADTs : Queues

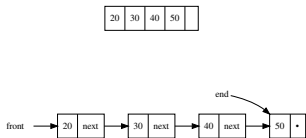
FIFO (First in, First out):



- Intuitively more “useful” than a stack.
- Think of implementing any kind of service (printer, web etc.)
- Operations include enqueue, dequeue and size.

# ADTs : Queues

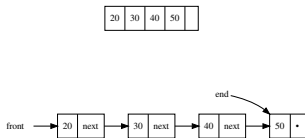
FIFO (First in, First out):



- Intuitively more “useful” than a stack.
- Think of implementing any kind of service (printer, web etc.)
- Operations include enqueue, dequeue and size.

# ADTs : Queues

FIFO (First in, First out):



- Intuitively more “useful” than a stack.
- Think of implementing any kind of service (printer, web etc.)
- Operations include enqueue, dequeue and size.

queue.h

```
1  #pragma once
2
3  #include "../General/general.h"
4
5  typedef int queue_type;
6
7  typedef struct queue queue;
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <string.h>
12 #include <assert.h>
13
14 /* Create an empty queue */
15 queue* queue_init(void);
16 /* Add element on end */
17 void queue_enqueue(queue* q, queue_type v);
18 /* Take element off front */
19 bool queue_dequeue(queue* q, queue_type* d);
20 /* Return size of queue */
21 int queue_size(queue* q);
22 /* Clears all space used */
23 bool queue_free(queue* q);
24
25 /* Helps with visualisation & testing */
26 void queue_tostring(queue* q, char* str);
```



# ADTs : Queues (Fixed) I

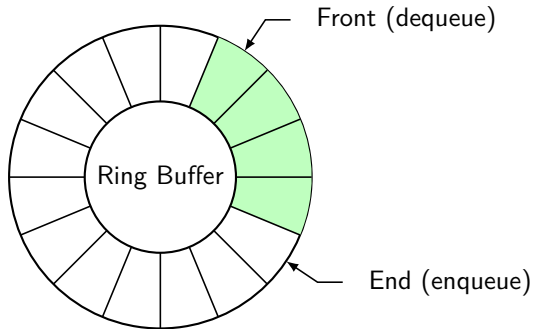
specific.h

```
1  #pragma once
2
3  #define FORMATSTR "%d"
4  #define ELEMSIZE 20
5
6  #define QUEUETYPE "Fixed"
7
8  #define BOUNDED 5000
9
10 struct queue {
11     /* Underlying array */
12     queuetype a[BOUNDED];
13     int front;
14     int end;
15 };
16
17 #define DOTFILE 5000
```

# ADTs : Queues (Fixed) I

specific.h

```
1  #pragma once
2
3  #define FORMATSTR "%d"
4  #define ELEMSIZE 20
5
6  #define QUEUETYPE "Fixed"
7
8  #define BOUNDED 5000
9
10 struct queue {
11     /* Underlying array */
12     queuetype a[BOUNDED];
13     int front;
14     int end;
15 };
16
17 #define DOTFILE 5000
```



# ADTs : Queues (Fixed) II

## fixed.c

```
1  #include "../queue.h"
2  #include "specific.h"
3
4  void __inc(int* p);
5
6  queue* queue_init(void)
7  {
8      queue* q = (queue*) ncalloc(1, sizeof(queue));
9      return q;
10 }
11
12
13 void queue_enqueue(queue* q, queuetype d)
14 {
15     if(q){
16         q->a[q->end] = d;
17         __inc(&q->end);
18         if(q->end == q->front){
19             on_error("Queue too large");
20         }
21     }
22 }
```

# ADTs : Queues (Fixed) II

## fixed.c

```
1  #include "../queue.h"
2  #include "specific.h"
3
4  void _inc(int* p);
5
6  queue* queue_init(void)
7  {
8      queue* q = (queue*) ncalloc(1, sizeof(queue));
9      return q;
10 }
11
12
13 void queue_enqueue(queue* q, queuetype d)
14 {
15     if(q){
16         q->a[q->end] = d;
17         _inc(&q->end);
18         if(q->end == q->front){
19             on_error("Queue too large");
20         }
21     }
22 }
```

```
1  bool queue_dequeue(queue* q, queuetype* d)
2  {
3      if((q==NULL) || (q->front==q->end)){
4          return false;
5      }
6      *d = q->a[q->front];
7      _inc(&q->front);
8      return true;
9  }
10
11 void queue_tostring(queue* q, char* str)
12 {
13     char tmp[ELEMSIZE];
14     str[0] = '\0';
15     if((q==NULL) || (queue_size(q)==0)){
16         return;
17     }
18     for(int i=q->front; i != q->end;){
19         sprintf(tmp, FORMATSTR, q->a[i]);
20         strcat(str, tmp);
21         strcat(str, "|");
22         _inc(&i);
23     }
24     str[strlen(str)-1] = '\0';
25 }
```

# ADTs : Queues (Fixed) III

```
1  int queue_size(queue* q)
2  {
3      if(q==NULL){
4          return 0;
5      }
6      if(q->end >= q->front){
7          return q->end-q->front;
8      }
9      return q->end + BOUNDED - q->front;
10 }
11
12 bool queue_free(queue* q)
13 {
14     free(q);
15     return true;
16 }
17
18 void __inc(int* p)
19 {
20     *p = (*p + 1) % BOUNDED;
21 }
```

# ADTs : Queues (Fixed) III

```
1  int queue_size(queue* q)
2  {
3      if(q==NULL){
4          return 0;
5      }
6      if(q->end >= q->front){
7          return q->end - q->front;
8      }
9      return q->end + BOUNDED - q->front;
10 }
11
12 bool queue_free(queue* q)
13 {
14     free(q);
15     return true;
16 }
17
18 void __inc(int* p)
19 {
20     *p = (*p + 1) % BOUNDED;
21 }
```

- We need a thorough testing program

# ADTs : Queues (Fixed) III

```
1  int queue_size(queue* q)
2  {
3      if(q==NULL){
4          return 0;
5      }
6      if(q->end >= q->front){
7          return q->end - q->front;
8      }
9      return q->end + BOUNDED - q->front;
10 }
11
12 bool queue_free(queue* q)
13 {
14     free(q);
15     return true;
16 }
17
18 void __inc(int* p)
19 {
20     *p = (*p + 1) % BOUNDED;
21 }
```

- We need a thorough testing program
- We'll see queues again for traversing trees

# ADTs : Queues (Fixed) III

```
1  int queue_size(queue* q)
2  {
3      if(q==NULL){
4          return 0;
5      }
6      if(q->end >= q->front){
7          return q->end - q->front;
8      }
9      return q->end + BOUNDED - q->front;
10 }
11
12 bool queue_free(queue* q)
13 {
14     free(q);
15     return true;
16 }
17
18 void __inc(int* p)
19 {
20     *p = (*p + 1) % BOUNDED;
21 }
```

- We need a thorough testing program
- We'll see queues again for traversing trees
- Simulating a (slow) printer



# ADTs : Queues (Linked) I

## specific.h

```
1  #pragma once
2
3  #define FORMATSTR "%d"
4  #define ELEMSIZE 20
5
6  #define QUEUETYPE "Linked"
7
8  struct dataframe {
9      queuetype i;
10     struct dataframe* next;
11 };
12 typedef struct dataframe dataframe;
13
14 struct queue {
15     /* Underlying array */
16     dataframe* front;
17     dataframe* end;
18     int size;
19 };
```

# ADTs : Queues (Linked) I

## specific.h

```
1  #pragma once
2
3  #define FORMATSTR "%d"
4  #define ELEMSIZE 20
5
6  #define QUEUETYPE "Linked"
7
8  struct dataframe {
9      queuetype i;
10     struct dataframe* next;
11 };
12 typedef struct dataframe dataframe;
13
14 struct queue {
15     /* Underlying array */
16     dataframe* front;
17     dataframe* end;
18     int size;
19 };
```

## linked.c

```
1  #include "../queue.h"
2  #include "specific.h"
3
4  queue* queue_init(void)
5  {
6      queue* q = (queue*) nalloc(1, sizeof(queue));
7      return q;
8  }
9
10 void queue_enqueue(queue* q, queuetype d)
11 {
12     dataframe* f;
13     if(q == NULL){
14         return;
15     }
16
17     /* Copy the data */
18     f = nalloc(1, sizeof(dataframe));
19     f->i = d;
20
21     /* 1st one */
22     if(q->front == NULL){
23         q->front = f;
24         q->end = f;
25         q->size = q->size + 1;
26         return;
27     }
28     /* Not 1st */
29     q->end->next = f;
30     q->end = f;
31     q->size = q->size + 1;
32 }
```

# ADTs : Queues (Linked) II

```
1  bool queue_dequeue(queue* q, queue_t* d)
2  {
3      dataframe* f;
4      if((q==NULL) || (q->front==NULL) || (q->end==NULL)){
5          return false;
6      }
7      f = q->front->next;
8      *d = q->front->i;
9      free(q->front);
10     q->front = f;
11     q->size = q->size - 1;
12     return true;
13 }
14
15 bool queue_free(queue* q)
16 {
17     if(q){
18         dataframe* tmp;
19         dataframe* p = q->front;
20         while(p!=NULL){
21             tmp = p->next;
22             free(p);
23             p = tmp;
24         }
25         free(q);
26     }
27     return true;
28 }
```

# ADTs : Queues (Linked) II

```
1  bool queue_dequeue(queue* q, queuetype* d)
2  {
3      dataframe* f;
4      if((q==NULL) || (q->front==NULL) || (q->end==NULL)){
5          return false;
6      }
7      f = q->front->next;
8      *d = q->front->i;
9      free(q->front);
10     q->front = f;
11     q->size = q->size - 1;
12     return true;
13 }
14
15 bool queue_free(queue* q)
16 {
17     if(q){
18         dataframe* tmp;
19         dataframe* p = q->front;
20         while(p!=NULL){
21             tmp = p->next;
22             free(p);
23             p = tmp;
24         }
25         free(q);
26     }
27     return true;
28 }
```

```
1  void queue_tostring(queue* q, char* str)
2  {
3      dataframe *p;
4      char tmp[ELEMSIZE];
5      str[0] = '\0';
6      if((q==NULL) || (q->front == NULL)){
7          return;
8      }
9      p = q->front;
10     while(p){
11         sprintf(tmp, FORMATSTR, p->i);
12         strcat(str, tmp);
13         strcat(str, "|");
14         p = p->next;
15     }
16     str[strlen(str)-1] = '\0';
17 }
18
19 int queue_size(queue* q)
20 {
21     if((q==NULL) || (q->front==NULL)){
22
23         return 0;
24     }
25     return q->size;
26 }
```

## Detour : Graphviz

- There exists a nice package, called Graphviz:

```
sudo apt install graphviz
```

# Detour : Graphviz

- There exists a nice package, called Graphviz:
- This allows the visualisation of graphs/dynamic structures using the simple .dot language:

```
digraph {  
    a -> b; b -> c; c -> a;  
}
```

# Detour : Graphviz

- There exists a nice package, called Graphviz:

```
sudo apt install graphviz
```

- This allows the visualisation of graphs/dynamic structures using the simple .dot language:

```
digraph {  
    a -> b; b -> c; c -> a;  
}
```

- To create a .pdf:

```
dot -Tpdf -o graphviz.pdf examp1.dot
```

# Detour : Graphviz

- There exists a nice package, called Graphviz:

```
sudo apt install graphviz
```

- This allows the visualisation of graphs/dynamic structures using the simple .dot language:

```
digraph {  
    a -> b; b -> c; c -> a;  
}
```

- To create a .pdf:

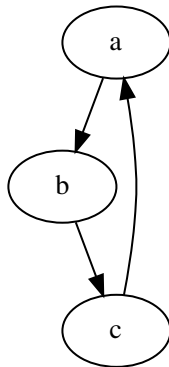
```
dot -Tpdf -o graphviz.pdf examp1.dot
```



# Detour : Graphviz

- There exists a nice package, called Graphviz:  
`sudo apt install graphviz`
- This allows the visualisation of graphs/dynamic structures using the simple .dot language:  

```
digraph {  
    a -> b; b -> c; c -> a;  
}
```
- To create a .pdf:  
`dot -Tpdf -o graphviz.pdf examp1.dot`



# Table of Contents

N : Recursion

O : Algorithms I - Search

P : Linked Data Structures

Q : ADTs - Collection

R : ADTs - Stacks

S : ADTs - Queues

**T : ADTs - Trees**

U : ADTs - Hashing

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

# Binary Trees : Data Structures

- Binary trees are used extensively in computer science

# Binary Trees : Data Structures

- Binary trees are used extensively in computer science
- Game Trees

# Binary Trees : Data Structures

- Binary trees are used extensively in computer science
- Game Trees
- Searching

# Binary Trees : Data Structures

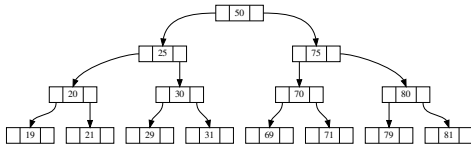
- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting

# Binary Trees : Data Structures

- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting

# Binary Trees : Data Structures

- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting

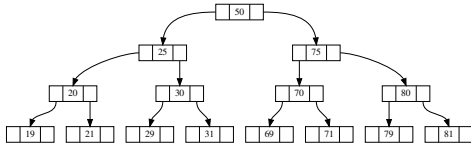




# Binary Trees : Data Structures

- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting

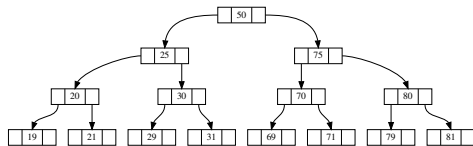
- Trees drawn upside-down !



# Binary Trees : Data Structures

- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting

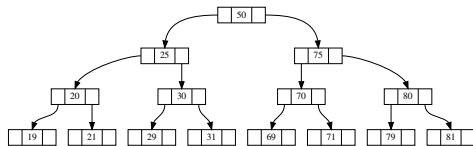
- Trees drawn upside-down !
- Ancestor relationships: '50' is the parent of '25' and '75'.



# Binary Trees : Data Structures

- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting

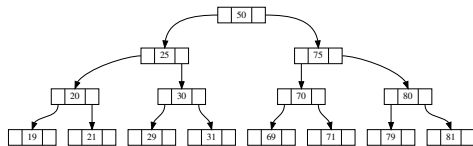
- Trees drawn upside-down !
- Ancestor relationships: '50' is the parent of '25' and '75'.
- Can refer to left and right children



# Binary Trees : Data Structures

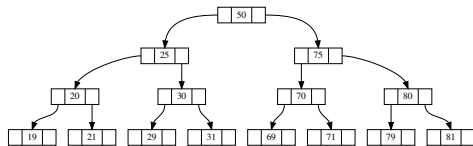
- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting

- Trees drawn upside-down !
- Ancestor relationships: '50' is the parent of '25' and '75'.
- Can refer to left and right children
- In a tree, there is only one path from the root to any child



# Binary Trees : Data Structures

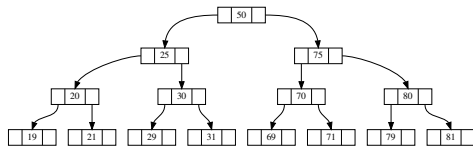
- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting



- Trees drawn upside-down !
- Ancestor relationships: '50' is the parent of '25' and '75'.
- Can refer to left and right children
- In a tree, there is only one path from the root to any child
- A node with no children is a leaf

# Binary Trees : Data Structures

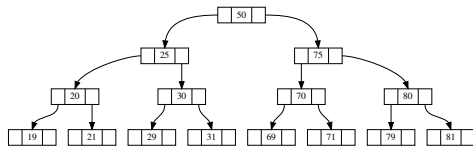
- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting



- Trees drawn upside-down !
- Ancestor relationships: '50' is the parent of '25' and '75'.
- Can refer to left and right children
- In a tree, there is only one path from the root to any child
- A node with no children is a leaf
- Most trees need to be created dynamically

# Binary Trees : Data Structures

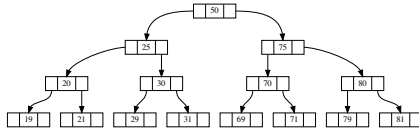
- Binary trees are used extensively in computer science
- Game Trees
- Searching
- Sorting



- Trees drawn upside-down !
- Ancestor relationships: '50' is the parent of '25' and '75'.
- Can refer to left and right children
- In a tree, there is only one path from the root to any child
- A node with no children is a leaf
- Most trees need to be created dynamically
- Empty subtrees are set to NULL

# Binary Search Trees

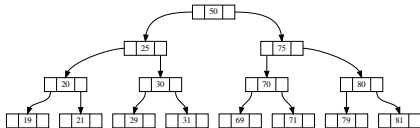
In a binary search tree the left-hand tree of a parent contains all keys less than the parent node, and the right-hand side all the keys greater than the parent node.





# Binary Search Trees

In a binary search tree the left-hand tree of a parent contains all keys less than the parent node, and the right-hand side all the keys greater than the parent node.



## bst.h

```
1  #include "../General/general.h"
2  #include "../Queue/queue.h"
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <assert.h>
7
8  bst* bst_init(void);
9
10 /* Insert 1 item into the tree */
11 bool bst_insert(bst* b, treetype d);
12
13 /* Return number of nodes in tree */
14 int bst_size(bst* b);
15
16 /* Whether the data d is stored in the tree */
17 bool bst_isin(bst* b, treetype d);
18
19 /* Bulk insert n items from an array a into an initialised tree */
20 bool bst_insertarray(bst* b, treetype* a, int n);
21
22 /* Clear all memory associated with tree, & set pointer to NULL */
23 bool bst_free(bst* b);
24
25 /* Optional ? */
26
27 char* bst_preorder(bst* b);
28 void bst_printlevel(bst* b);
29 /* Create string with tree as ((head)(left)(right)) */
30 char* bst_printlisp(bst* b);
31 /* Use Graphviz via a .dot file */
32 void bst_todot(bst* b, char* dotname);
```

# Binary Search Trees : Linked I

specific.h

```
1  #include <string.h>
2
3  typedef int treetype;
4  #define FORMATSIR "%i"
5  #define ELEMSIZE 20
6  #define BSTTYPE "Linked"
7
8  struct dataframe {
9      treetype d;
10     struct dataframe* left;
11     struct dataframe* right;
12 };
13 typedef struct dataframe dataframe;
14
15 struct bst {
16     dataframe* top;
17     /* Data element size, in bytes */
18 };
19 typedef struct bst bst;
```

# Binary Search Trees : Linked I

## specific.h

```
1  #include <string.h>
2
3  typedef int treetype;
4  #define FORMATSIR "%i"
5  #define ELEMSIZE 20
6  #define BSTTYPE "Linked"
7
8  struct dataframe {
9      treetype d;
10     struct dataframe* left;
11     struct dataframe* right;
12 };
13 typedef struct dataframe dataframe;
14
15 struct bst {
16     dataframe* top;
17     /* Data element size, in bytes */
18 };
19 typedef struct bst bst;
```

```
/* Based on geekforgeeks.org */
dataframe* __insert(dataframe* t, treetype d)
{
    dataframe* f;
    /* If the tree is empty, return a new frame */
    if (t == NULL){
        f = calloc(sizeof(dataframe), 1);
        f->d = d;
        return f;
    }
    /* Otherwise, recurs down the tree */
    if (d < t->d){
        t->left = __insert(t->left, d);
    }
    else if(d > t->d){
        t->right = __insert(t->right, d);
    }
    /* return the (unchanged) dataframe pointer */
    return t;
}
```

# Binary Search Trees : Linked II

```
bool __isin(dataframe* t, treetype d)
{
    if(t==NULL){
        return false;
    }
    if(t->d == d){
        return true;
    }
    if(d < t->d){
        return __isin(t->left, d);
    }
    else{
        return __isin(t->right, d);
    }
    return false;
}
```

# Binary Search Trees : Linked II

```
bool __isin(dataframe* t, treetype d)
{
    if(t==NULL){
        return false;
    }
    if(t->d == d){
        return true;
    }
    if(d < t->d){
        return __isin(t->left, d);
    }
    else{
        return __isin(t->right, d);
    }
    return false;
}
```

```
char* __printlisp(dataframe* t)
{
    char tmp[ELEMSIZE];
    char *s1, *s2, *p;

    if(t==NULL){
        /* \0 string */
        p = ncalloc(1,1);
        return p;
    }
    sprintf(tmp, FORMATSTR, t->d);
    s1 = __printlisp(t->left);
    s2 = __printlisp(t->right);
    p = ncalloc(strlen(s1)+strlen(s2)+strlen(tmp)+
        strlen("()() "), 1);
    sprintf(p, "%s(%s)(%s)", tmp, s1, s2);
    free(s1);
    free(s2);
    return p;
}
```

# Binary Trees using Arrays ?

- Don't rush to assume a linked data structure must be used to implement trees.

# Binary Trees using Arrays ?

- Don't rush to assume a linked data structure must be used to implement trees.
- You could use 1 cell of an array for the first node, the next two cells for its children, the next 4 cells for their children and so on.

# Binary Trees using Arrays ?

- Don't rush to assume a linked data structure must be used to implement trees.
- You could use 1 cell of an array for the first node, the next two cells for its children, the next 4 cells for their children and so on.
- You need to mark which cells are in use & which aren't ...



# Binary Trees using Arrays ?

- Don't rush to assume a linked data structure must be used to implement trees.
- You could use 1 cell of an array for the first node, the next two cells for its children, the next 4 cells for their children and so on.
- You need to mark which cells are in use & which aren't ...

# Binary Trees using Arrays ?

- Don't rush to assume a linked data structure must be used to implement trees.
- You could use 1 cell of an array for the first node, the next two cells for its children, the next 4 cells for their children and so on.
- You need to mark which cells are in use & which aren't ...

Counting from cell 1, for a tree with  $n$  nodes:

| To find                  | Use      | Iff             |
|--------------------------|----------|-----------------|
| The root                 | $A[1]$   | $A$ is nonempty |
| The left child of $A[i]$ | $A[2i]$  | $2i \leq n$     |
| The parent of $A[i]$     | $A[i/2]$ | $i > 1$         |
| Is $A[i]$ a leaf ?       | True     | $2i > n$        |

# Binary Search Trees : Realloc

## specific.h

```
1  #include <stdbool.h>
2
3  typedef int treetype;
4  #define FORMATSTR "%i"
5  #define ELEMSIZE 20
6  #define BSTTYPE "Realloc"
7
8  // Probably (2^n) -1
9  #define INITSIZE 31
10 #define SCALEFACTOR 2
11
12 struct dataframe {
13     treetype d;
14     bool isValid;
15 };
16 typedef struct dataframe dataframe;
17
18 struct bst {
19     dataframe* a;
20     int capacity;
21 };
22 typedef struct bst bst;
```

# Binary Search Trees : Realloc

## specific.h

```
1  #include <stdbool.h>
2
3  typedef int treetype;
4  #define FORMATSTR "%i"
5  #define ELEMSIZE 20
6  #define BSTTYPE "Realloc"
7
8  // Probably (2^n) -1
9  #define INITSIZE 31
10 #define SCALEFACTOR 2
11
12 struct dataframe {
13     treetype d;
14     bool invalid;
15 };
16 typedef struct dataframe dataframe;
17
18 struct bst {
19     dataframe* a;
20     int capacity;
21 };
22 typedef struct bst bst;
```

## Using a queue for Level-Order traversal:

```
void bst_printlevel(bst* b)
{
    treetype n;
    if((b==NULL) || (! __isvalid(b, 0))){
        return;
    }
    /* Make a queue of cell indices */
    queue* q = queue_init();
    queue_enqueue(q, 0);
    while(queue_dequeue(q, &n) && __isvalid(b, (int)n)){
        printf(FORMATSTR, b->a[n].d);
        putchar(' ');
        queue_enqueue(q, __leftchild((int)n));
        queue_enqueue(q, __rightchild((int)n));
    }
}
```

# Binary Search Trees : Complexity

- So, in a nicely balanced tree, insertion, deletion and search are all  $O(\log n)$ .

# Binary Search Trees : Complexity

- So, in a nicely balanced tree, insertion, deletion and search are all  $O(\log n)$ .
- But: if the root of the tree is not well chosen, or the keys to be inserted are ordered, the tree can become a linked list !

# Binary Search Trees : Complexity

- So, in a nicely balanced tree, insertion, deletion and search are all  $O(\log n)$ .
- But: if the root of the tree is not well chosen, or the keys to be inserted are ordered, the tree can become a linked list !
- In this case, complexity becomes  $O(n)$ .

# Binary Search Trees : Complexity

- So, in a nicely balanced tree, insertion, deletion and search are all  $O(\log n)$ .
- But: if the root of the tree is not well chosen, or the keys to be inserted are ordered, the tree can become a linked list !
- In this case, complexity becomes  $O(n)$ .
- The tree search performs best when well balanced trees are formed.



# Binary Search Trees : Complexity

- So, in a nicely balanced tree, insertion, deletion and search are all  $O(\log n)$ .
- But: if the root of the tree is not well chosen, or the keys to be inserted are ordered, the tree can become a linked list !
- In this case, complexity becomes  $O(n)$ .
- The tree search performs best when well balanced trees are formed.
- Large body of literature about creating & re-balancing trees - Red-Black trees, Tries, 2-3 trees, AVL trees etc.

# Table of Contents

N : Recursion

O : Algorithms I - Search

P : Linked Data Structures

Q : ADTs - Collection

R : ADTs - Stacks

S : ADTs - Queues

T : ADTs - Trees

**U : ADTs - Hashing**

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:  
`xx-##-##-##-x`

# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:  
`xx-##-##-##-x`
- There are 17.6 billion combinations (around  $2^{34}$ ).

# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:  
`xx-##-##-##-x`
- There are 17.6 billion combinations (around  $2^{34}$ ).
- Could use an array of 17.6 billion entries, which would make searching for a particular entry trivial !

# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:  
`xx-##-##-##-x`
- There are 17.6 billion combinations (around  $2^{34}$ ).
- Could use an array of 17.6 billion entries, which would make searching for a particular entry trivial !
- Especially wasteful since only our (5000) employees need to be stored.

# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:  
`xx-##-##-##-x`
- There are 17.6 billion combinations (around  $2^{34}$ ).
- Could use an array of 17.6 billion entries, which would make searching for a particular entry trivial !
- Especially wasteful since only our (5000) employees need to be stored.

# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:  
xx-##-##-##-x
- There are 17.6 billion combinations (around  $2^{34}$ ).
- Could use an array of 17.6 billion entries, which would make searching for a particular entry trivial !
- Especially wasteful since only our (5000) employees need to be stored.
- Here we examine a method that, using an array of 6000 elements, would require 2.1 comparisons on average.



# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:

xx-##-##-##-x

- There are 17.6 billion combinations (around  $2^{34}$ ).
  - Could use an array of 17.6 billion entries, which would make searching for a particular entry trivial !
  - Especially wasteful since only our (5000) employees need to be stored.
- Here we examine a method that, using an array of 6000 elements, would require 2.1 comparisons on average.
  - A hash function is a mapping,  $h(K)$ , that maps from key  $K$ , onto the index of an entry.

# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:

xx-##-##-##-x

- There are 17.6 billion combinations (around  $2^{34}$ ).
  - Could use an array of 17.6 billion entries, which would make searching for a particular entry trivial !
  - Especially wasteful since only our (5000) employees need to be stored.
- Here we examine a method that, using an array of 6000 elements, would require 2.1 comparisons on average.
  - A hash function is a mapping,  $h(K)$ , that maps from key  $K$ , onto the index of an entry.
  - A black-box into which we insert a key (e.g. NI number) and out pops an array index.

# ADTs : Hashing

- To keep records of employees we might index (search) them by using their National Insurance number:

xx-##-##-##-x

- There are 17.6 billion combinations (around  $2^{34}$ ).
- Could use an array of 17.6 billion entries, which would make searching for a particular entry trivial !
- Especially wasteful since only our (5000) employees need to be stored.

- Here we examine a method that, using an array of 6000 elements, would require 2.1 comparisons on average.
- A hash function is a mapping,  $h(K)$ , that maps from key  $K$ , onto the index of an entry.
- A black-box into which we insert a key (e.g. NI number) and out pops an array index.
- As an example lets use an array of size 11 to store some airport codes, e.g. PEK, BRS, FRA.

# Hashing : Airport Codes

- In a three letter string  $X_2X_1X_0$  the letter 'A' has the value 0, 'B' has the value 1 etc.

# Hashing : Airport Codes

- In a three letter string  $X_2X_1X_0$  the letter 'A' has the value 0, 'B' has the value 1 etc.
- One hash function is:

$$h(K) = (X_2 * 26^2 + X_1 * 26 + X_0) \% 11$$

# Hashing : Aipor Codes

- In a three letter string  $X_2X_1X_0$  the letter 'A' has the value 0, 'B' has the value 1 etc.
- One hash function is:

$$h(K) = (X_2 * 26^2 + X_1 * 26 + X_0) \% 11$$

- Applying this to "DCA":

$$h("DCA") =$$

$$(3 * 26^2 + 2 * 26 + 0) \% 11$$

$$h("DCA") = (2080) \% 11$$

$$h("DCA") = 1$$

# Hashing : Aipor Codes

- In a three letter string  $X_2X_1X_0$  the letter 'A' has the value 0, 'B' has the value 1 etc.
- One hash function is:

$$h(K) = (X_2 * 26^2 + X_1 * 26 + X_0) \% 11$$

- Applying this to "DCA":

$$h("DCA") =$$

$$(3 * 26^2 + 2 * 26 + 0) \% 11$$

$$h("DCA") = (2080) \% 11$$

$$h("DCA") = 1$$

# Hashing : Aiport Codes

- In a three letter string  $X_2X_1X_0$  the letter 'A' has the value 0, 'B' has the value 1 etc.

- One hash function is:

$$h(K) = (X_2 * 26^2 + X_1 * 26 + X_0) \% 11$$

- Applying this to "DCA":

$$h("DCA") =$$

$$(3 * 26^2 + 2 * 26 + 0) \% 11$$

$$h("DCA") = (2080) \% 11$$

$$h("DCA") = 1$$

- Inserting "PHL", "ORY" and "GCM":

|     |    |
|-----|----|
|     | 0  |
|     | 1  |
|     | 2  |
|     | 3  |
| PHL | 4  |
|     | 5  |
| GCM | 6  |
|     | 7  |
| ORY | 8  |
|     | 9  |
|     | 10 |



# Hashing : Aiport Codes

- In a three letter string  $X_2X_1X_0$  the letter 'A' has the value 0, 'B' has the value 1 etc.
- One hash function is:

$$h(K) = (X_2 * 26^2 + X_1 * 26 + X_0) \% 11$$

- Applying this to "DCA":

$$h("DCA") =$$

$$(3 * 26^2 + 2 * 26 + 0) \% 11$$

$$h("DCA") = (2080) \% 11$$

$$h("DCA") = 1$$

- Inserting "PHL", "ORY" and "GCM":

|     |    |
|-----|----|
|     | 0  |
|     | 1  |
|     | 2  |
|     | 3  |
| PHL | 4  |
|     | 5  |
| GCM | 6  |
|     | 7  |
| ORY | 8  |
|     | 9  |
|     | 10 |

- However, inserting "HKG" causes a collision.

|   |    |
|---|----|
|   | 0  |
|   | 1  |
|   | 2  |
|   | 3  |
| ? | 4  |
|   | 5  |
|   | 6  |
|   | 7  |
|   | 8  |
|   | 9  |
|   | 10 |

HKG

# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.

# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.
- Collisions occur when a hash function maps two different keys onto the same address.

# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.
- Collisions occur when a hash function maps two different keys onto the same address.
- It's very difficult to choose 'good' hashing functions.

# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.
- Collisions occur when a hash function maps two different keys onto the same address.
- It's very difficult to choose 'good' hashing functions.
- Collisions are common - the **von Mises** paradox. When 23 keys are randomly mapped onto 365 addresses there is a 50% chance of a collision.

# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.
- Collisions occur when a hash function maps two different keys onto the same address.
- It's very difficult to choose 'good' hashing functions.
- Collisions are common - the **von Mises** paradox. When 23 keys are randomly mapped onto 365 addresses there is a 50% chance of a collision.

# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.
- Collisions occur when a hash function maps two different keys onto the same address.
- It's very difficult to choose 'good' hashing functions.
- Collisions are common - the **von Mises** paradox. When 23 keys are randomly mapped onto 365 addresses there is a 50% chance of a collision.
- The policy of finding another free location if a collision occurs is called open-addressing.

# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.
- Collisions occur when a hash function maps two different keys onto the same address.
- It's very difficult to choose 'good' hashing functions.
- Collisions are common - the **von Mises** paradox. When 23 keys are randomly mapped onto 365 addresses there is a 50% chance of a collision.
- The policy of finding another free location if a collision occurs is called open-addressing.
- If a collision occurs then keep stepping backwards (with wrap-around) until a free location is encountered.



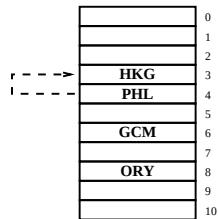
# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.
- Collisions occur when a hash function maps two different keys onto the same address.
- It's very difficult to choose 'good' hashing functions.
- Collisions are common - the **von Mises** paradox. When 23 keys are randomly mapped onto 365 addresses there is a 50% chance of a collision.
- The policy of finding another free location if a collision occurs is called open-addressing.
- If a collision occurs then keep stepping backwards (with wrap-around) until a free location is encountered.

# Hashing : Collisions

- An ideal hashing function maps keys into the array in a *uniform* and *random* manner.
- Collisions occur when a hash function maps two different keys onto the same address.
- It's very difficult to choose 'good' hashing functions.
- Collisions are common - the **von Mises** paradox. When 23 keys are randomly mapped onto 365 addresses there is a 50% chance of a collision.

- The policy of finding another free location if a collision occurs is called open-addressing.
- If a collision occurs then keep stepping backwards (with wrap-around) until a free location is encountered.



# Double Hashing

- This simple method of open-addressing is linear-probing.

# Double Hashing

- This simple method of open-addressing is linear-probing.
- The step taken each time (probe decrement) need not be 1.

# Double Hashing

- This simple method of open-addressing is linear-probing.
- The step taken each time (probe decrement) need not be 1.
- Open-addressing through use of linear-probing is a very simple technique, double-hashing is generally much more successful.

# Double Hashing

- This simple method of open-addressing is linear-probing.
- The step taken each time (probe decrement) need not be 1.
- Open-addressing through use of linear-probing is a very simple technique, double-hashing is generally much more successful.
- A second function  $p(K)$  decides the size of the probe decrement.

# Double Hashing

- This simple method of open-addressing is linear-probing.
- The step taken each time (probe decrement) need not be 1.
- Open-addressing through use of linear-probing is a very simple technique, double-hashing is generally much more successful.
- A second function  $p(K)$  decides the size of the probe decrement.

# Double Hashing

- This simple method of open-addressing is linear-probing.
- The step taken each time (probe decrement) need not be 1.
- Open-addressing through use of linear-probing is a very simple technique, double-hashing is generally much more successful.
- A second function  $p(K)$  decides the size of the probe decrement.
- The function is chosen so that two keys which collide at the same address will have different probe decrements, e.g. :

$$p(K) = \text{MAX}(1, ((X_2 * 26^2 + X_1 * 26 + X_0)/11)\%11)$$



# Double Hashing

- This simple method of open-addressing is linear-probing.
- The step taken each time (probe decrement) need not be 1.
- Open-addressing through use of linear-probing is a very simple technique, double-hashing is generally much more successful.
- A second function  $p(K)$  decides the size of the probe decrement.

- The function is chosen so that two keys which collide at the same address will have different probe decrements, e.g. :

$$p(K) = \text{MAX}(1, ((X_2 * 26^2 + X_1 * 26 + X_0)/11)\%11)$$

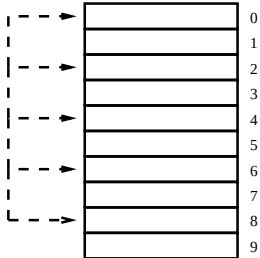
- Although "PHL" and "HKG" share the same primary hash value of  $h(K) = 4$ , they have different probe decrements:

$$p(\text{"PHL"}) = 4$$

$$p(\text{"HKG"}) = 3$$

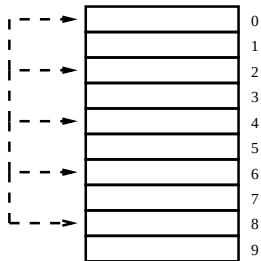
# Hashing : Primes and Chaining

- If the size of our array,  $M$ , was even and the probe decrement was chosen to be 2, then only half of the locations could be probed.



# Hashing : Primes and Chaining

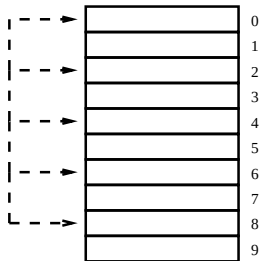
- If the size of our array,  $M$ , was even and the probe decrement was chosen to be 2, then only half of the locations could be probed.



- Often we choose our table size to be a prime number and our probe decrement to be a number in the range  $1 \dots M - 1$ .

# Hashing : Primes and Chaining

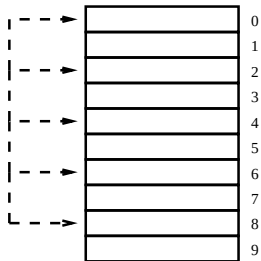
- If the size of our array,  $M$ , was even and the probe decrement was chosen to be 2, then only half of the locations could be probed.



- Often we choose our table size to be a prime number and our probe decrement to be a number in the range  $1 \dots M - 1$ .

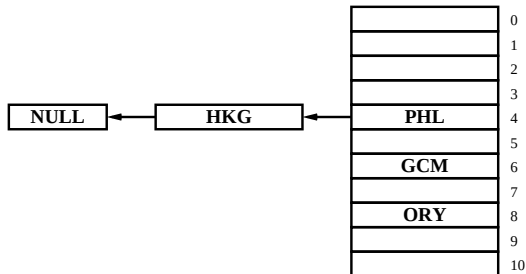
# Hashing : Primes and Chaining

- If the size of our array,  $M$ , was even and the probe decrement was chosen to be 2, then only half of the locations could be probed.



- Often we choose our table size to be a prime number and our probe decrement to be a number in the range  $1 \dots M - 1$ .

Open-addressing is not the only method of collision reduction. Another common one is separate chaining.



# A Practical Hash Function

```
1  #include <stdio.h>
2
3  int hash(unsigned int sz, char *s);
4
5  int main(void)
6  {
7
8      char str[] = "Hello World!";
9      // Hash modulus 7919
10     printf("%d\n", hash(7919, str));
11     return 0;
12 }
13
14
15 /*
16  Modified Bernstein hashing
17  5381 & 33 are magic numbers required by the algorithm
18  */
19 int hash(unsigned int sz, char *s)
20 {
21     unsigned long hash = 5381;
22     int c;
23     while((c = (*s++))) {
24         hash = 33 * hash ^ c;
25     }
26     return (int)(hash%sz);
27 }
```

Execution :

5479

# A Practical Hash Function

```
1  #include <stdio.h>
2
3  int hash(unsigned int sz, char *s);
4
5  int main(void)
6  {
7
8      char str[] = "Hello World!";
9      // Hash modulus 7919
10     printf("%d\n", hash(7919, str));
11     return 0;
12 }
13
14 /*
15  Modified Bernstein hashing
16  5381 & 33 are magic numbers required by the algorithm
17  */
18 int hash(unsigned int sz, char *s)
19 {
20     unsigned long hash = 5381;
21     int c;
22     while((c = (*s++))) {
23         hash = 33 * hash ^ c;
24     }
25     return (int)(hash%sz);
26 }
27 }
```

Execution :

5479

Has similarities to the implementation of rand() :

```
int rand_r(unsigned int* seed);

int main(void)
{
    unsigned int seed = 0;
    printf("%d\n", rand_r(&seed));
    return 0;
}

/* This algorithm is mentioned in the ISO C standard,
   here extended for 32 bits. */
int rand_r(unsigned int* seed)
{
    unsigned int next = *seed;
    int result;
    next *= 1103515245;
    next += 12345;
    result = (unsigned int) (next / 65536) % 2048;
    next *= 1103515245;
    next += 12345;
    result <<= 10;
    result ^= (unsigned int) (next / 65536) % 1024;
    next *= 1103515245;
    next += 12345;
    result <<= 10;
    result ^= (unsigned int) (next / 65536) % 1024;
    *seed = next;
    return result;
}
```

Execution :

1012484

# Cuckoo Hashing

- We have two tables, each with their **own** hash function.

```
Empty: copied farandoles into table 0(4)
Empty: copied bronzine into table 0(12)
Empty: copied auscultatory into table 0(5)
Empty: copied bifer into table 0(13)
Empty: copied steepgrass into table 0(6)
Empty: copied prevised into table 0(7)
Empty: copied oomph into table 0(8)
empodium, so cuckooed out auscultatory from table 0(5)
Empty: copied auscultatory into table 1(10)
interquarreled, so cuckooed out bronzine from table 0(12)
Empty: copied bronzine into table 1(5)
ranseur, so cuckooed out empodium from table 0(5)
Empty: copied empodium into table 1(4)
Empty: copied megalodon into table 0(11)
geosynchronous, so cuckooed out megalodon from table 0(11)
Empty: copied megalodon into table 1(14)
Empty: copied osmeteria into table 0(14)
Table getting full -> rehashed old sz =16
```



# Cuckoo Hashing

- We have two tables, each with their **own** hash function.
- We only need to check two cells when searching.

```
Empty: copied farandoles into table 0(4)
Empty: copied bronzine into table 0(12)
Empty: copied auscultatory into table 0(5)
Empty: copied bifer into table 0(13)
Empty: copied steepgrass into table 0(6)
Empty: copied prevised into table 0(7)
Empty: copied oomph into table 0(8)
empodium, so cuckooed out auscultatory from table 0(5)
Empty: copied auscultatory into table 1(10)
interquarreled, so cuckooed out bronzine from table 0(12)
Empty: copied bronzine into table 1(5)
ranseur, so cuckooed out empodium from table 0(5)
Empty: copied empodium into table 1(4)
Empty: copied megalodon into table 0(11)
geosynchronous, so cuckooed out megalodon from table 0(11)
Empty: copied megalodon into table 1(14)
Empty: copied osmeteria into table 0(14)
Table getting full -> rehashed old sz =16
```

# Cuckoo Hashing

- We have two tables, each with their **own** hash function.
- We only need to check two cells when searching.
- On collision, the existing item is 'cuckooed' out of it's cell into the other table.

```
Empty: copied farandoles into table 0(4)
Empty: copied bronzine into table 0(12)
Empty: copied auscultatory into table 0(5)
Empty: copied bifer into table 0(13)
Empty: copied steepgrass into table 0(6)
Empty: copied prevised into table 0(7)
Empty: copied oomph into table 0(8)
empodium, so cuckooed out auscultatory from table 0(5)
Empty: copied auscultatory into table 1(10)
interquarreled, so cuckooed out bronzine from table 0(12)
Empty: copied bronzine into table 1(5)
ranseur, so cuckooed out empodium from table 0(5)
Empty: copied empodium into table 1(4)
Empty: copied megalodon into table 0(11)
geosynchronous, so cuckooed out megalodon from table 0(11)
Empty: copied megalodon into table 1(14)
Empty: copied osmeteria into table 0(14)
Table getting full -> rehashed old sz =16
```

# Cuckoo Hashing

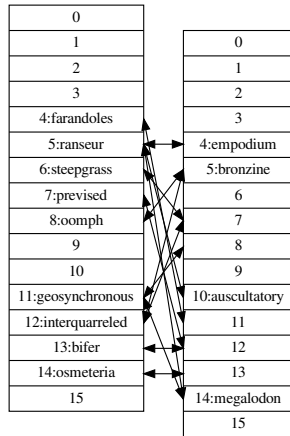
- We have two tables, each with their **own** hash function.
- We only need to check two cells when searching.
- On collision, the existing item is 'cuckooed' out of it's cell into the other table.

```
Empty: copied farandoles into table 0(4)
Empty: copied bronzine into table 0(12)
Empty: copied auscultatory into table 0(5)
Empty: copied bifer into table 0(13)
Empty: copied steepgrass into table 0(6)
Empty: copied prevised into table 0(7)
Empty: copied oomph into table 0(8)
empodium, so cuckooed out auscultatory from table 0(5)
Empty: copied auscultatory into table 1(10)
interquarreled, so cuckooed out bronzine from table 0(12)
Empty: copied bronzine into table 1(5)
ranseur, so cuckooed out empodium from table 0(5)
Empty: copied empodium into table 1(4)
Empty: copied megalodon into table 0(11)
geosynchronous, so cuckooed out megalodon from table 0(11)
Empty: copied megalodon into table 1(14)
Empty: copied osmeteria into table 0(14)
Table getting full -> rehashed old sz =16
```

# Cuckoo Hashing

- We have two tables, each with their **own** hash function.
- We only need to check two cells when searching.
- On collision, the existing item is 'cuckooed' out of it's cell into the other table.

Empty: copied farandoles into table 0(4)  
Empty: copied bronzine into table 0(12)  
Empty: copied auscultatory into table 0(5)  
Empty: copied bifer into table 0(13)  
Empty: copied steepgrass into table 0(6)  
Empty: copied prevised into table 0(7)  
Empty: copied oomph into table 0(8)  
empodium, so cuckooed out auscultatory from table 0(5)  
Empty: copied auscultatory into table 1(10)  
interquarreled, so cuckooed out bronzine from table 0(12)  
Empty: copied bronzine into table 1(5)  
ranseur, so cuckooed out empodium from table 0(5)  
Empty: copied empodium into table 1(4)  
Empty: copied megalodon into table 0(11)  
geosynchronous, so cuckooed out megalodon from table 0(11)  
Empty: copied megalodon into table 1(14)  
Empty: copied osmeteria into table 0(14)  
Table getting full -> rehashed old sz =16



# Table of Contents

N : Recursion

O : Algorithms I - Search

P : Linked Data Structures

Q : ADTs - Collection

R : ADTs - Stacks

S : ADTs - Queues

T : ADTs - Trees

U : ADTs - Hashing

**V : Algorithms II - Sort**

W : Algorithms III - Huffman/Strings

# Algorithms : Sorting

```
#define NUMS 6

void bubble_sort(int b[], int s);

int main(void)
{
    int a[] = {3, 4, 1, 2, 9, 0};
    bubble_sort(a, NUMS);
    for(int i=0; i<NUMS; i++){
        printf("%i ", a[i]);
    }
    printf("\n");
    return 0;
}

void bubble_sort(int b[], int s)
{
    bool changes;
    do{
        changes = false;
        for(int i=0; i<s-1; i++){
            if(b[i] > b[i+1]){
                SWAP(b[i], b[i+1]);
                changes = true;
            }
        }
    }while(changes);
}
```

Execution :

0 1 2 3 4 9

# Algorithms : Sorting

```
#define NUMS 6

void bubble_sort(int b[], int s);

int main(void)
{
    int a[] = {3, 4, 1, 2, 9, 0};
    bubble_sort(a, NUMS);
    for(int i=0; i<NUMS; i++){
        printf("%i ", a[i]);
    }
    printf("\n");
    return 0;
}

void bubble_sort(int b[], int s)
{
    bool changes;
    do{
        changes = false;
        for(int i=0; i<s-1; i++){
            if(b[i] > b[i+1]){
                SWAP(b[i], b[i+1]);
                changes = true;
            }
        }
    }while(changes);
}
```

Execution :

0 1 2 3 4 9

- Bubblesort has complexity  $O(n^2)$ , therefore very inefficient.

# Algorithms : Sorting

```
#define NUMS 6

void bubble_sort(int b[], int s);

int main(void)
{
    int a[] = {3, 4, 1, 2, 9, 0};
    bubble_sort(a, NUMS);
    for(int i=0; i<NUMS; i++){
        printf("%i ", a[i]);
    }
    printf("\n");
    return 0;
}

void bubble_sort(int b[], int s)
{
    bool changes;
    do{
        changes = false;
        for(int i=0; i<s-1; i++){
            if(b[i] > b[i+1]){
                SWAP(b[i], b[i+1]);
                changes = true;
            }
        }
    }while(changes);
}
```

Execution :

0 1 2 3 4 9

- Bubblesort has complexity  $O(n^2)$ , therefore very inefficient.
- If an algorithm uses comparison keys to decide the correct order then the theoretical lower bound on complexity is  $O(n \log n)$ . From wiki:



# Algorithms : Sorting

```
#define NUMS 6

void bubble_sort(int b[], int s);

int main(void)
{
    int a[] = {3, 4, 1, 2, 9, 0};
    bubble_sort(a, NUMS);
    for(int i=0; i<NUMS; i++){
        printf("%i ", a[i]);
    }
    printf("\n");
    return 0;
}

void bubble_sort(int b[], int s)
{
    bool changes;
    do{
        changes = false;
        for(int i=0; i<s-1; i++){
            if(b[i] > b[i+1]){
                SWAP(b[i], b[i+1]);
                changes = true;
            }
        }
    }while(changes);
}
```

Execution :

0 1 2 3 4 9

- Bubblesort has complexity  $O(n^2)$ , therefore very inefficient.
- If an algorithm uses comparison keys to decide the correct order then the theoretical lower bound on complexity is  $O(n \log n)$ . From wiki:

# Algorithms : Sorting

```
#define NUMS 6

void bubble_sort(int b[], int s);

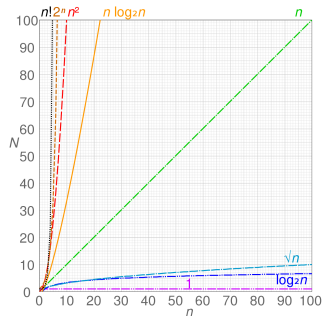
int main(void)
{
    int a[] = {3, 4, 1, 2, 9, 0};
    bubble_sort(a, NUMS);
    for(int i=0; i<NUMS; i++){
        printf("%i ", a[i]);
    }
    printf("\n");
    return 0;
}

void bubble_sort(int b[], int s)
{
    bool changes;
    do{
        changes = false;
        for(int i=0; i<s-1; i++){
            if(b[i] > b[i+1]){
                SWAP(b[i], b[i+1]);
                changes = true;
            }
        }
    } while(changes);
}
```

Execution :

0 1 2 3 4 9

- Bubblesort has complexity  $O(n^2)$ , therefore very inefficient.
- If an algorithm uses comparison keys to decide the correct order then the theoretical lower bound on complexity is  $O(n \log n)$ . From wiki:



# Algorithms : Merge Sort

- Transposition (Bubblesort)

# Algorithms : Merge Sort

- Transposition (Bubblesort)
- Insertion Sort (Lab Work)

# Algorithms : Merge Sort

- Transposition (Bubblesort)
- Insertion Sort (Lab Work)
- Priority Queue (Selection sort, Heap sort)

# Algorithms : Merge Sort

- Transposition (Bubblesort)
- Insertion Sort (Lab Work)
- Priority Queue (Selection sort, Heap sort)
- Divide & Conquer (Merge & Quick sorts)

# Algorithms : Merge Sort

- Transposition (Bubblesort)
- Insertion Sort (Lab Work)
- Priority Queue (Selection sort, Heap sort)
- Divide & Conquer (Merge & Quick sorts)
- Address Calculation (Proxmap)

# Algorithms : Merge Sort

- Transposition (Bubblesort)
- Insertion Sort (Lab Work)
- Priority Queue (Selection sort, Heap sort)
- Divide & Conquer (Merge & Quick sorts)
- Address Calculation (Proxmap)



# Algorithms : Merge Sort

- Transposition (Bubblesort)
- Insertion Sort (Lab Work)
- Priority Queue (Selection sort, Heap sort)
- Divide & Conquer (Merge & Quick sorts)
- Address Calculation (Proxmap)
- Merge sort is divide-and-conquer in that you divide the array into two halves, mergesort each half and then merge the two halves into order.

# Algorithms : Merge Sort

- Transposition (Bubblesort)
- Insertion Sort (Lab Work)
- Priority Queue (Selection sort, Heap sort)
- Divide & Conquer (Merge & Quick sorts)
- Address Calculation (Proxmap)
- Merge sort is divide-and-conquer in that you divide the array into two halves, mergesort each half and then merge the two halves into order.

# Algorithms : Merge Sort

- Transposition (Bubblesort)
- Insertion Sort (Lab Work)
- Priority Queue (Selection sort, Heap sort)
- Divide & Conquer (Merge & Quick sorts)
- Address Calculation (Proxmap)

- Merge sort is divide-and-conquer in that you divide the array into two halves, mergesort each half and then merge the two halves into order.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  void mergesort(int *src, int *spare, int l, int r);
6  void merge(int *src, int *spare, int l, int m, int r);
7
8  #define NUM 5000
9
10 int main(void)
11 {
12     int a[NUM];
13     int spare[NUM];
14
15     for(int i=0; i<NUM; i++){
16         a[i] = rand()%100;
17     }
18
19     mergesort(a, spare, 0, NUM-1);
20
21     for(int i=0; i<NUM; i++){
22         printf("%4d ==> %d\n", i, a[i]);
23     }
24
25     return 0;
26 }
```

# Merge Sort II

```
void mergesort(int *src, int *spare, int l, int r)
{
    int m = (l+r)/2;
    if(l != r){
        mergesort(src, spare, l, m);
        mergesort(src, spare, m+1, r);
        merge(src, spare, l, m, r);
    }
}

void merge(int *src, int *spare, int l, int m, int r)
{
    int s1 = l;
    int s2 = m+1;
    int d = l;

    do{
        if(src[s1] < src[s2]){
            spare[d++] = src[s1++];
        }
        else{
            spare[d++] = src[s2++];
        }
    }while((s1 <= m) && (s2 <= r));

    if(s1 > m){
        memcpy(&spare[d], &src[s2], sizeof(spare[0])*(r-s2+1));
    }
    else{
        memcpy(&spare[d], &src[s1], sizeof(spare[0])*(m-s1+1));
    }
    memcpy(&src[l], &spare[l], (r-l+1)*sizeof(spare[0]));
}
```

# Merge Sort II

```
void mergesort(int *src, int *spare, int l, int r)
{
    int m = (l+r)/2;
    if(l != r){
        mergesort(src, spare, l, m);
        mergesort(src, spare, m+1, r);
        merge(src, spare, l, m, r);
    }
}

void merge(int *src, int *spare, int l, int m, int r)
{
    int s1 = l;
    int s2 = m+1;
    int d = l;

    do{
        if(src[s1] < src[s2]){
            spare[d++] = src[s1++];
        }
        else{
            spare[d++] = src[s2++];
        }
    }while((s1 <= m) && (s2 <= r));

    if(s1 > m){
        memcpy(&spare[d], &src[s2], sizeof(spare[0])*(r-s2+1));
    }
    else{
        memcpy(&spare[d], &src[s1], sizeof(spare[0])*(m-s1+1));
    }
    memcpy(&src[l], &spare[l], (r-l+1)*sizeof(spare[0]));
}
```

- Quicksort is also divide-and-conquer.

# Merge Sort II

```
void mergesort(int *src, int *spare, int l, int r)
{
    int m = (l+r)/2;
    if(l != r){
        mergesort(src, spare, l, m);
        mergesort(src, spare, m+1, r);
        merge(src, spare, l, m, r);
    }
}

void merge(int *src, int *spare, int l, int m, int r)
{
    int s1 = l;
    int s2 = m+1;
    int d = l;

    do{
        if(src[s1] < src[s2]){
            spare[d++] = src[s1++];
        }
        else{
            spare[d++] = src[s2++];
        }
    }while((s1 <= m) && (s2 <= r));

    if(s1 > m){
        memcpy(&spare[d], &src[s2], sizeof(spare[0])*(r-s2+1));
    }
    else{
        memcpy(&spare[d], &src[s1], sizeof(spare[0])*(m-s1+1));
    }
    memcpy(&src[l], &spare[l], (r-l+1)*sizeof(spare[0]));
}
```

- Quicksort is also divide-and-conquer.
- Choose some value in the array as the *pivot* key.

# Merge Sort II

```
void mergesort(int *src, int *spare, int l, int r)
{
    int m = (l+r)/2;
    if(l != r){
        mergesort(src, spare, l, m);
        mergesort(src, spare, m+1, r);
        merge(src, spare, l, m, r);
    }
}

void merge(int *src, int *spare, int l, int m, int r)
{
    int s1 = l;
    int s2 = m+1;
    int d = l;

    do{
        if(src[s1] < src[s2]){
            spare[d++] = src[s1++];
        }
        else{
            spare[d++] = src[s2++];
        }
    }while((s1 <= m) && (s2 <= r));

    if(s1 > m){
        memcpy(&spare[d], &src[s2], sizeof(spare[0])*(r-s2+1));
    }
    else{
        memcpy(&spare[d], &src[s1], sizeof(spare[0])*(m-s1+1));
    }
    memcpy(&src[l], &spare[l], (r-l+1)*sizeof(spare[0]));
}
```

- Quicksort is also divide-and-conquer.
- Choose some value in the array as the *pivot* key.
- This key is used to divide the array into two partitions. The left partition contains keys  $\leq$  pivot key, the right partition contains keys  $>$  pivot.

# Merge Sort II

```
void mergesort(int *src, int *spare, int l, int r)
{
    int m = (l+r)/2;
    if(l != r){
        mergesort(src, spare, l, m);
        mergesort(src, spare, m+1, r);
        merge(src, spare, l, m, r);
    }
}

void merge(int *src, int *spare, int l, int m, int r)
{
    int s1 = l;
    int s2 = m+1;
    int d = l;

    do{
        if(src[s1] < src[s2]){
            spare[d++] = src[s1++];
        }
        else{
            spare[d++] = src[s2++];
        }
    }while((s1 <= m) && (s2 <= r));

    if(s1 > m){
        memcpy(&spare[d], &src[s2], sizeof(spare[0])*(r-s2+1));
    }
    else{
        memcpy(&spare[d], &src[s1], sizeof(spare[0])*(m-s1+1));
    }
    memcpy(&src[l], &spare[l], (r-l+1)*sizeof(spare[0]));
}
```

- Quicksort is also divide-and-conquer.
- Choose some value in the array as the *pivot* key.
- This key is used to divide the array into two partitions. The left partition contains keys  $\leq$  pivot key, the right partition contains keys  $>$  pivot.
- Once again, the sort is then applied recursively.



# Algorithms : Quicksort

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  int partition(int *a, int l, int r);
6  void quicksort(int *a, int l, int r);
7
8  #define NUM 100000
9
10 int main(void)
11 {
12     int a[NUM];
13
14     for(int i=0; i<NUM; i++){
15         a[i] = rand()%100;
16     }
17     quicksort(a, 0, NUM-1);
18
19     return 0;
20 }
21
22 void quicksort(int *a, int l, int r)
23 {
24     int pivpoint = partition(a, l, r);
25     if(l < pivpoint){
26         quicksort(a, l, pivpoint-1);
27     }
28     if(r > pivpoint){
29         quicksort(a, pivpoint+1, r);
30     }
31 }
```

# Algorithms : Quicksort

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  int partition(int *a, int l, int r);
6  void quicksort(int *a, int l, int r);
7
8  #define NUM 100000
9
10 int main(void)
11 {
12     int a[NUM];
13
14     for(int i=0; i<NUM; i++){
15         a[i] = rand()%100;
16     }
17     quicksort(a, 0, NUM-1);
18
19     return 0;
20 }
21
22 void quicksort(int *a, int l, int r)
23 {
24     int pivpoint = partition(a, l, r);
25     if(l < pivpoint){
26         quicksort(a, l, pivpoint-1);
27     }
28     if(r > pivpoint){
29         quicksort(a, pivpoint+1, r);
30     }
31 }
```

```
int partition(int *a, int l, int r)
{
    int piv = a[l];
    while(l<r){
        /* Right -> Left Scan */
        while(piv < a[r] && l<r) r--;
        if(r!=l){
            a[l] = a[r];
            l++;
        }
        /* Left -> Right Scan */
        while(piv > a[l] && l<r) l++;
        if(r!=l){
            a[r] = a[l];
            r--;
        }
    }
    a[r] = piv;
    return r;
}
```

# qsort()

- Theoretically both methods have a complexity  $O(n \log n)$

# qsort()

- Theoretically both methods have a complexity  $O(n \log n)$
- Quicksort is preferred because it requires less memory and is generally faster.

# qsort()

- Theoretically both methods have a complexity  $O(n \log n)$
- Quicksort is preferred because it requires less memory and is generally faster.
- Quicksort can go badly wrong if the pivot key chosen is either the maximum or minimum value in the array.

# qsort()

- Theoretically both methods have a complexity  $O(n \log n)$
- Quicksort is preferred because it requires less memory and is generally faster.
- Quicksort can go badly wrong if the pivot key chosen is either the maximum or minimum value in the array.
- Quicksort is so loved by programmers that a library version of it exists in ANSI C.

# qsort()

- Theoretically both methods have a complexity  $O(n \log n)$
- Quicksort is preferred because it requires less memory and is generally faster.
- Quicksort can go badly wrong if the pivot key chosen is either the maximum or minimum value in the array.
- Quicksort is so loved by programmers that a library version of it exists in ANSI C.
- If you need an off-the-shelf sort, this is often a good option.

# qsort()

- Theoretically both methods have a complexity  $O(n \log n)$
- Quicksort is preferred because it requires less memory and is generally faster.
- Quicksort can go badly wrong if the pivot key chosen is either the maximum or minimum value in the array.
- Quicksort is so loved by programmers that a library version of it exists in ANSI C.
- If you need an off-the-shelf sort, this is often a good option.
- Interestingly, `bsearch()` also exists in `stdlib.h` - needs a sorted array.



# qsort()

- Theoretically both methods have a complexity  $O(n \log n)$
- Quicksort is preferred because it requires less memory and is generally faster.
- Quicksort can go badly wrong if the pivot key chosen is either the maximum or minimum value in the array.
- Quicksort is so loved by programmers that a library version of it exists in ANSI C.
- If you need an off-the-shelf sort, this is often a good option.
- Interestingly, `bsearch()` also exists in `stdlib.h` - needs a sorted array.

# qsort()

- Theoretically both methods have a complexity  $O(n \log n)$
- Quicksort is preferred because it requires less memory and is generally faster.
- Quicksort can go badly wrong if the pivot key chosen is either the maximum or minimum value in the array.
- Quicksort is so loved by programmers that a library version of it exists in ANSI C.
- If you need an off-the-shelf sort, this is often a good option.
- Interestingly, bsearch() also exists in stdlib.h - needs a sorted array.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int intcompare(const void *a, const void *b);
5
6  int main(void)
7  {
8      int a[10];
9
10     for(int i=0; i<10; i++){
11         a[i] = 9 - i;
12     }
13
14     qsort(a, 10, sizeof(int), intcompare);
15
16     for (int i=0; i<10; i++){
17         printf(" %d",a[i]);
18     }
19     printf("\n");
20     return 0;
21 }
22
23
24 int intcompare(const void *a, const void *b)
25 {
26     const int *ia = (const int *)a;
27     const int *ib = (const int *)b;
28     return *ia - *ib;
29 }
```

# Algorithms : The Radix Sort

- The radix sort is also known as the bin sort, a name derived from its origin as a technique used on (now obsolete) card sorters.

# Algorithms : The Radix Sort

- The radix sort is also known as the bin sort, a name derived from its origin as a technique used on (now obsolete) card sorters.
- For integer data, repeated passes of radix sort focus on the right digit (the units), then the second digit (the tens) and so on.

# Algorithms : The Radix Sort

- The radix sort is also known as the bin sort, a name derived from its origin as a technique used on (now obsolete) card sorters.
- For integer data, repeated passes of radix sort focus on the right digit (the units), then the second digit (the tens) and so on.
- Strings could be sorted in a similar manner.

# Algorithms : The Radix Sort

- The radix sort is also known as the bin sort, a name derived from its origin as a technique used on (now obsolete) card sorters.
- For integer data, repeated passes of radix sort focus on the right digit (the units), then the second digit (the tens) and so on.
- Strings could be sorted in a similar manner.

# Algorithms : The Radix Sort

- The radix sort is also known as the bin sort, a name derived from its origin as a technique used on (now obsolete) card sorters.
- For integer data, repeated passes of radix sort focus on the right digit (the units), then the second digit (the tens) and so on.
- Strings could be sorted in a similar manner.

459 254 472 534 649 239 432 654 477

0

1

2 472 432

3

4 254 534 654

5

6

7 477

8

9 459 649 239

Read out the new list:

472 432 254 534 654 477 459 649 239

# Radix Sort II

472 432 254 534 654 477 459 649 239

**0**

**1**

**2**

**3** 432 534 239

**4** 649

**5** 254 654 459

**6**

**7** 472 477

**8**

**9**

432 534 239 649 254 654 459 472 477



# Radix Sort II

472 432 254 534 654 477 459 649 239

**0**

**1**

**2**

**3** 432 534 239

**4** 649

**5** 254 654 459

**6**

**7** 472 477

**8**

**9**

432 534 239 649 254 654 459 472 477

432 534 239 649 254 654 459 472 477

**0**

**1**

**2** 239 254

**3**

**4** 432 459 472 477

**5** 534

**6** 649 654

**7**

**8**

**9**

239 254 432 459 472 477 534 649 654

# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .

# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .
- It is difficult to write an all-purpose radix sort - you need a different one for doubles, integers, strings etc.

# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .
- It is difficult to write an all-purpose radix sort - you need a different one for doubles, integers, strings etc.
- $O(n)$  simply means that the number of operations can be bounded by  $k.n$ , for some constant  $k$ .

# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .
- It is difficult to write an all-purpose radix sort - you need a different one for doubles, integers, strings etc.
- $O(n)$  simply means that the number of operations can be bounded by  $k.n$ , for some constant  $k$ .
- With the radix sort,  $k$  is often very large.

# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .
- It is difficult to write an all-purpose radix sort - you need a different one for doubles, integers, strings etc.
- $O(n)$  simply means that the number of operations can be bounded by  $k.n$ , for some constant  $k$ .
- With the radix sort,  $k$  is often very large.
- For many lists this may be less efficient than more traditional  $O(n \log n)$  algorithms.

# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .
- It is difficult to write an all-purpose radix sort - you need a different one for doubles, integers, strings etc.
- $O(n)$  simply means that the number of operations can be bounded by  $k.n$ , for some constant  $k$ .
- With the radix sort,  $k$  is often very large.
- For many lists this may be less efficient than more traditional  $O(n \log n)$  algorithms.
- Sometimes you'll want to profile your code.

# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .
- It is difficult to write an all-purpose radix sort - you need a different one for doubles, integers, strings etc.
- $O(n)$  simply means that the number of operations can be bounded by  $k.n$ , for some constant  $k$ .
- With the radix sort,  $k$  is often very large.
- For many lists this may be less efficient than more traditional  $O(n \log n)$  algorithms.
- Sometimes you'll want to profile your code.
- Compile with the `-pg` flag.



# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .
- It is difficult to write an all-purpose radix sort - you need a different one for doubles, integers, strings etc.
- $O(n)$  simply means that the number of operations can be bounded by  $k.n$ , for some constant  $k$ .
- With the radix sort,  $k$  is often very large.
- For many lists this may be less efficient than more traditional  $O(n \log n)$  algorithms.
- Sometimes you'll want to profile your code.
- Compile with the `-pg` flag.
- Executing your code produces a `gmon.out` file.

# Radix Sort Discussion and gprof

- This has a theoretical complexity of  $O(n)$ .
- It is difficult to write an all-purpose radix sort - you need a different one for doubles, integers, strings etc.
- $O(n)$  simply means that the number of operations can be bounded by  $k.n$ , for some constant  $k$ .
- With the radix sort,  $k$  is often very large.
- For many lists this may be less efficient than more traditional  $O(n \log n)$  algorithms.
- Sometimes you'll want to profile your code.
- Compile with the `-pg` flag.
- Executing your code produces a `gmon.out` file.
- Now: `gprof ./executable gmon.out` shows the function-call profile of your code.

# Table of Contents

N : Recursion

O : Algorithms I - Search

P : Linked Data Structures

Q : ADTs - Collection

R : ADTs - Stacks

S : ADTs - Queues

T : ADTs - Trees

U : ADTs - Hashing

V : Algorithms II - Sort

W : Algorithms III - Huffman/Strings

# Algorithm : Huffman Compression

- Often we wish to compress data, to reduce storage requirements, or to speed transmission.

# Algorithm : Huffman Compression

- Often we wish to compress data, to reduce storage requirements, or to speed transmission.
- Text is particularly suited to compression since using one byte per character is wasteful - some letters occur much more frequently.

# Algorithm : Huffman Compression

- Often we wish to compress data, to reduce storage requirements, or to speed transmission.
- Text is particularly suited to compression since using one byte per character is wasteful - some letters occur much more frequently.
- Need to give frequently occurring letters short codes, typically a few bits. Less common letters can have long bit patterns.

# Algorithm : Huffman Compression

- Often we wish to compress data, to reduce storage requirements, or to speed transmission.
- Text is particularly suited to compression since using one byte per character is wasteful - some letters occur much more frequently.
- Need to give frequently occurring letters short codes, typically a few bits. Less common letters can have long bit patterns.
- To encode the string "BABBAGE":

# Algorithm : Huffman Compression

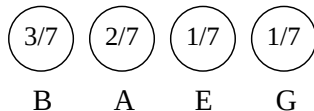
- Often we wish to compress data, to reduce storage requirements, or to speed transmission.
- Text is particularly suited to compression since using one byte per character is wasteful - some letters occur much more frequently.
- Need to give frequently occurring letters short codes, typically a few bits. Less common letters can have long bit patterns.
- To encode the string "BABBAGE":



# Algorithm : Huffman Compression

- Often we wish to compress data, to reduce storage requirements, or to speed transmission.
- Text is particularly suited to compression since using one byte per character is wasteful - some letters occur much more frequently.
- Need to give frequently occurring letters short codes, typically a few bits. Less common letters can have long bit patterns.

- To encode the string "BABBAGE":



- Keep a list of characters, ordered by their frequency

# Huffman Compression II

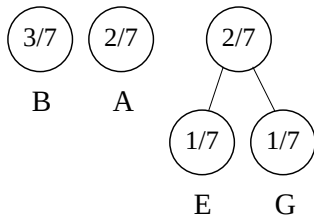
- Use the two least frequent to form a sub-tree, and re-order (sort) the nodes :

# Huffman Compression II

- Use the two least frequent to form a sub-tree, and re-order (sort) the nodes :

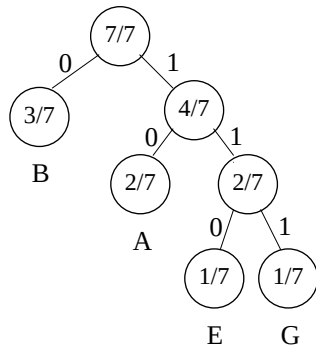
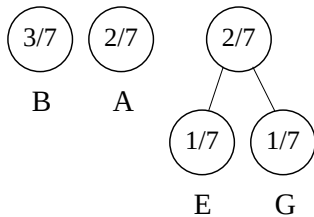
# Huffman Compression II

- Use the two least frequent to form a sub-tree, and re-order (sort) the nodes :



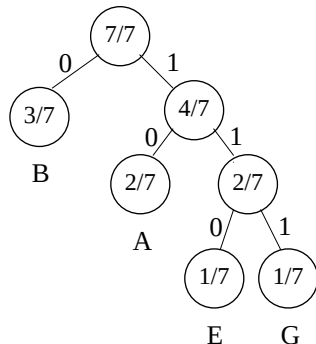
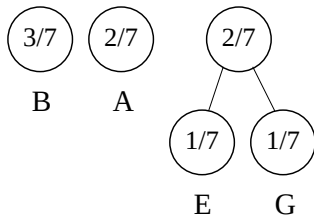
# Huffman Compression II

- Use the two least frequent to form a sub-tree, and re-order (sort) the nodes :



# Huffman Compression II

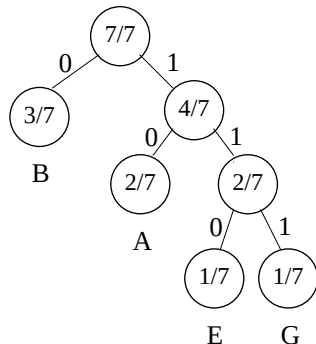
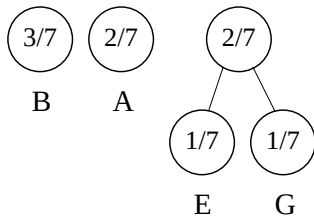
- Use the two least frequent to form a sub-tree, and re-order (sort) the nodes :



- $A = 10$ ,  $B = 0$ ,  $E = 110$ ,  $G = 111$

# Huffman Compression II

- Use the two least frequent to form a sub-tree, and re-order (sort) the nodes :



- A = 10, B = 0, E = 110, G = 111
- String stored using 13 bits.

# Algorithm : Rabin-Karp String Searching

- The task of searching for a string amongst a large amount of text is commonly required in word-processors, but more interestingly in massive Biological Databases e.g. searching for amino acids in protein sequences.



# Algorithm : Rabin-Karp String Searching

- The task of searching for a string amongst a large amount of text is commonly required in word-processors, but more interestingly in massive Biological Databases e.g. searching for amino acids in protein sequences.
- How difficult can it be ? Don't you just do a character by character brute-force search ?

Master String : AAAAAAAAAAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

# Algorithm : Rabin-Karp String Searching

- The task of searching for a string amongst a large amount of text is commonly required in word-processors, but more interestingly in massive Biological Databases e.g. searching for amino acids in protein sequences.
- How difficult can it be ? Don't you just do a character by character brute-force search ?

Master String : AAAAAAAAAAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

# Algorithm : Rabin-Karp String Searching

- The task of searching for a string amongst a large amount of text is commonly required in word-processors, but more interestingly in massive Biological Databases e.g. searching for amino acids in protein sequences.
- How difficult can it be ? Don't you just do a character by character brute-force search ?
- If the master string has  $m$  characters, and the search string has  $n$  characters then this search has complexity:  $O(mn)$

Master String : AAAAAAAAAAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

# Algorithm : Rabin-Karp String Searching

- The task of searching for a string amongst a large amount of text is commonly required in word-processors, but more interestingly in massive Biological Databases e.g. searching for amino acids in protein sequences.
- How difficult can it be ? Don't you just do a character by character brute-force search ?

Master String : AAAAAAAAAAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

- If the master string has  $m$  characters, and the search string has  $n$  characters then this search has complexity:  $O(mn)$
- Recall that to compute a hash function on a word we did something like:

$$h("NEILL") =$$

$$(13 \times 26^4 + 4 \times 26^3 + 8 \times 26^2 + 11 \times 26 + 11) \% P$$

where  $P$  is a big prime number.

# Algorithm : Rabin-Karp String Searching

- The task of searching for a string amongst a large amount of text is commonly required in word-processors, but more interestingly in massive Biological Databases e.g. searching for amino acids in protein sequences.
- How difficult can it be ? Don't you just do a character by character brute-force search ?

Master String : AAAAAAAAAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

Substring : AAAAAAH

- If the master string has  $m$  characters, and the search string has  $n$  characters then this search has complexity:  $O(mn)$
- Recall that to compute a hash function on a word we did something like:

$$h("NEILL") =$$

$$(13 \times 26^4 + 4 \times 26^3 + 8 \times 26^2 + 11 \times 26 + 11) \% P$$

where  $P$  is a big prime number.

- This can be expanded by Horner's method to:

$$(((((((13 \times 26) + 4) \times 26) + 8) \times 26) + 11) \times 26 + 11) \% P$$

# Rabin-Karp II

- For a large search string, overflow can occur. We therefore move the *mod* operation inside the brackets:

$$((((((13 \times 26) + 4) \% P \times 26) + 8) \% P \times 26) + 11) \% P \times 26 + 11) \% P$$

# Rabin-Karp II

- For a large search string, overflow can occur. We therefore move the *mod* operation inside the brackets:

$$((((((13 \times 26) + 4) \% P \times 26) + 8) \% P \times 26) + 11) \% P \times 26 + 11) \% P$$

- We can compute a hash number for the search string, and for the initial part of the master string.

# Rabin-Karp II

- For a large search string, overflow can occur. We therefore move the *mod* operation inside the brackets:

$$((((((13 \times 26) + 4) \% P \times 26) + 8) \% P \times 26) + 11) \% P \times 26 + 11) \% P$$

- We can compute a hash number for the search string, and for the initial part of the master string.
- When we compute the hash number for the next part of the master, most of the computation is common, we just need to take out the effect of the first letter and add in the effect of the new one.



# Rabin-Karp II

- For a large search string, overflow can occur. We therefore move the *mod* operation inside the brackets:

$$((((((13 \times 26) + 4) \% P \times 26) + 8) \% P \times 26) + 11) \% P \times 26 + 11) \% P$$

- We can compute a hash number for the search string, and for the initial part of the master string.
- When we compute the hash number for the next part of the master, most of the computation is common, we just need to take out the effect of the first letter and add in the effect of the new one.
- One small calculation each time we move one place right in the master.

# Rabin-Karp II

- For a large search string, overflow can occur. We therefore move the *mod* operation inside the brackets:

$$((((((13 \times 26) + 4) \% P \times 26) + 8) \% P \times 26) + 11) \% P \times 26 + 11) \% P$$

- We can compute a hash number for the search string, and for the initial part of the master string.
- When we compute the hash number for the next part of the master, most of the computation is common, we just need to take out the effect of the first letter and add in the effect of the new one.
- One small calculation each time we move one place right in the master.
- Complexity  $O(m + n)$  roughly, but need to check that two identical hash numbers really has identified two identical strings.

# Rabin-Karp II

- For a large search string, overflow can occur. We therefore move the *mod* operation inside the brackets:

$$((((((13 \times 26) + 4) \% P \times 26) + 8) \% P \times 26) + 11) \% P \times 26 + 11) \% P$$

- We can compute a hash number for the search string, and for the initial part of the master string.
- When we compute the hash number for the next part of the master, most of the computation is common, we just need to take out the effect of the first letter and add in the effect of the new one.
- One small calculation each time we move one place right in the master.
- Complexity  $O(m + n)$  roughly, but need to check that two identical hash numbers really has identified two identical strings.

# Rabin-Karp II

- For a large search string, overflow can occur. We therefore move the *mod* operation inside the brackets:

$$((((((13 \times 26) + 4) \% P \times 26) + 8) \% P \times 26) + 11) \% P \times 26 + 11) \% P$$

- We can compute a hash number for the search string, and for the initial part of the master string.
- When we compute the hash number for the next part of the master, most of the computation is common, we just need to take out the effect of the first letter and add in the effect of the new one.
- One small calculation each time we move one place right in the master.
- Complexity  $O(m + n)$  roughly, but need to check that two identical hash numbers really has identified two identical strings.

```
1  #include <string.h>
2  #include <assert.h>
3
4  #define Q 33554393
5  #define D 26
6  #define index(C) (C - 'A')
7
8  int rk(char *p, char *a);
9
10 int main(void)
11 {
12     assert(rk("STING",
13              "A STRING EXAMPLE CONSISTING OF ...") == 22);
14     return 0;
15 }
16
17 int rk(char *p, char *a)
18 {
19     int i, dM = 1, h1 = 0, h2 = 0;
20     int m = strlen(p);
21     int n = strlen(a);
22     for(i = 1; i < m; i++) dM = (D * dM) % Q;
23     for(i = 0; i < m; i++){
24         h1 = (h1 * D + index(p[i])) % Q;
25         h2 = (h2 * D + index(a[i])) % Q;
26     }
27     // h1 = search string hash, h2 = master string hash
28     for(i = 0; h1 != h2; i++){
29         h2 = (h2 + D * Q - index(a[i]) * dM) % Q;
30         h2 = (h2 + D + index(a[i + m])) % Q;
31         if(i > n - m) return n;
32     }
33     return i;
34 }
```

# Algorithm : Boyer-Moore String Searching

The Boyer-Moore algorithm uses (in part) an array flagging which characters form part of the search string and an array telling us how far to slide right if that character appears in the master and causes a mismatch.

Execution :

```
A STRING SEARCHING EXAMPLE CONSISTING OF ...  
  |      |  
STING    |  
    STING  
      STING
```

# Algorithm : Boyer-Moore String Searching

The Boyer-Moore algorithm uses (in part) an array flagging which characters form part of the search string and an array telling us how far to slide right if that character appears in the master and causes a mismatch.

Execution :

```
A STRING SEARCHING EXAMPLE CONSISTING OF ...  
  |      |  
STING    |  
      STING  
        STING
```

- With a right-to-left walk through the search string we see that the G and the R mismatch on the first comparison.

# Algorithm : Boyer-Moore String Searching

The Boyer-Moore algorithm uses (in part) an array flagging which characters form part of the search string and an array telling us how far to slide right if that character appears in the master and causes a mismatch.

Execution :

```
A STRING SEARCHING EXAMPLE CONSISTING OF ...  
  |      |  
STING    |  
      STING  
        STING
```

- With a right-to-left walk through the search string we see that the G and the R mismatch on the first comparison.
- Since R doesn't appear in the search string, we can take 5 steps to the right.

# Algorithm : Boyer-Moore String Searching

The Boyer-Moore algorithm uses (in part) an array flagging which characters form part of the search string and an array telling us how far to slide right if that character appears in the master and causes a mismatch.

Execution :

```
A STRING SEARCHING EXAMPLE CONSISTING OF ...
  |      |
STING    |
         |
        STING
          |
         STING
```

- With a right-to-left walk through the search string we see that the G and the R mismatch on the first comparison.
- Since R doesn't appear in the search string, we can take 5 steps to the right.
- The next comparison is between the G and the S. We can slide the search string right until it matches the S in the master.



# Boyer-Moore II

Execution :

A STRING SEARCHING EXAMPLE CONSISTING OF ...

|       |       |       |       |       |  |  |  |
|-------|-------|-------|-------|-------|--|--|--|
|       |       |       |       |       |  |  |  |
| STING |       |       |       |       |  |  |  |
|       | STING |       |       |       |  |  |  |
|       |       | STING |       |       |  |  |  |
|       |       |       | STING |       |  |  |  |
|       |       |       |       | STING |  |  |  |

- Now the C doesn't appear in the master and once again we can slide a full 5 places to the right.

# Boyer-Moore II

Execution :

A STRING SEARCHING EXAMPLE CONSISTING OF ...

|       |       |       |       |       |  |  |  |
|-------|-------|-------|-------|-------|--|--|--|
|       |       |       |       |       |  |  |  |
| STING |       |       |       |       |  |  |  |
|       | STING |       |       |       |  |  |  |
|       |       | STING |       |       |  |  |  |
|       |       |       | STING |       |  |  |  |
|       |       |       |       | STING |  |  |  |

- Now the C doesn't appear in the master and once again we can slide a full 5 places to the right.
- After 3 more full slides right we arrive at the T in CONSISTING.

# Boyer-Moore II

Execution :

```
A STRING SEARCHING EXAMPLE CONSISTING OF ...  
      |   |   |   |   |  
  STING |   |   |   |  
        |   |   |   |  
      STING |   |   |  
            |   |   |  
          STING |   |  
                |   |  
              STING
```

- Now the C doesn't appear in the master and once again we can slide a full 5 places to the right.
- After 3 more full slides right we arrive at the T in CONSISTING.
- We align the T's, and have found our match using 7 compares (plus 5 to verify the match).